



Multi-Threading







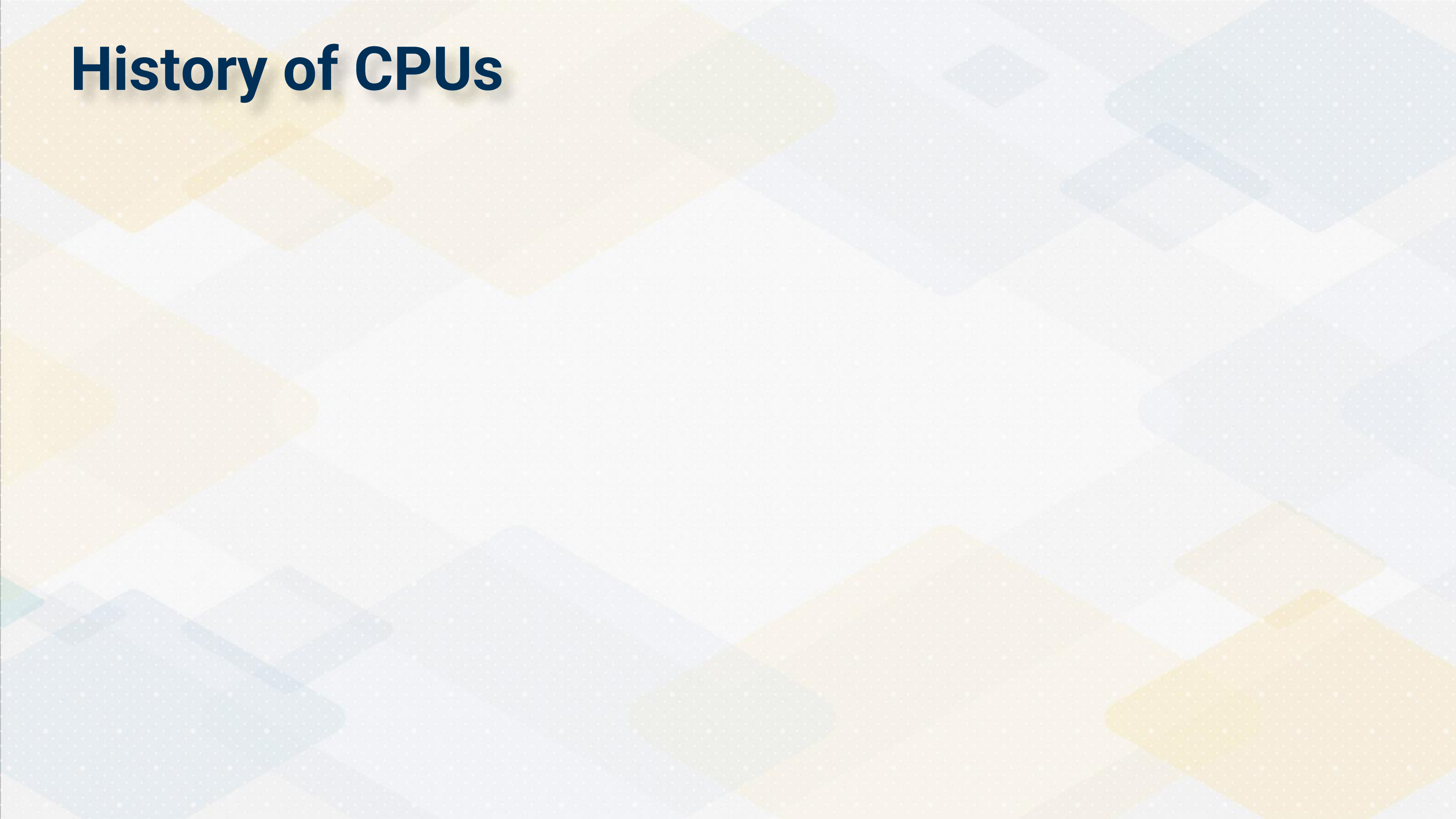
Multi-Core CPUs



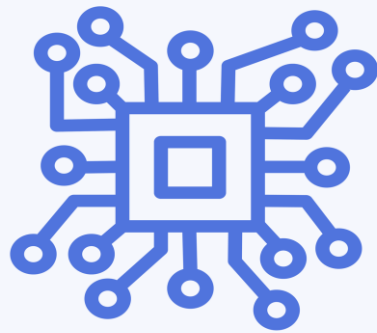
**Multi-Core
CPUs**

**Multi-
Threading
In C++**

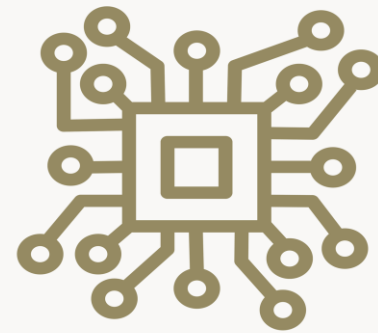
History of CPUs

The background of the slide is an abstract pattern of overlapping squares in various shades of yellow, blue, and green. The squares are arranged in a way that creates a sense of depth and movement, with some squares appearing more prominent than others. The overall effect is a modern, geometric design.

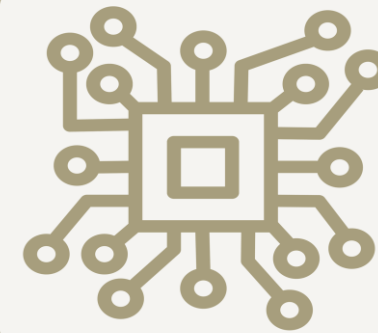
History of CPUs



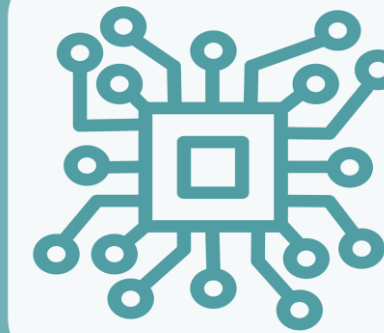
1970-1980s
First CPUs
Few
Megahertz



1990s
100s of
MHz to
over 1 GHz

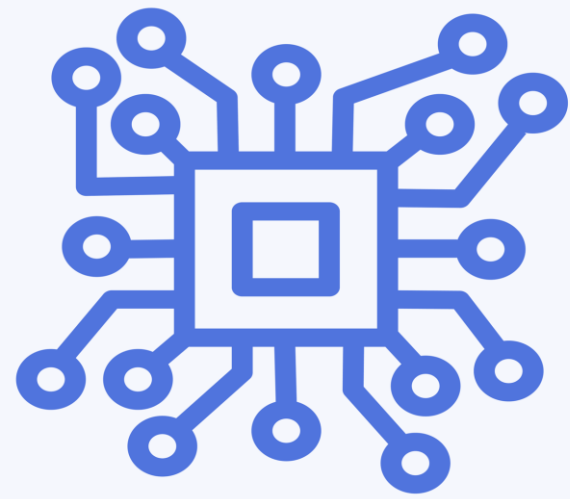


Early 2000s
Top-end
CPUs
over 3 GHz

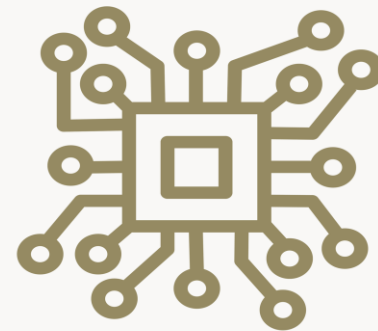


Mid 2000s
3.8 GHz
Thermal
Wall

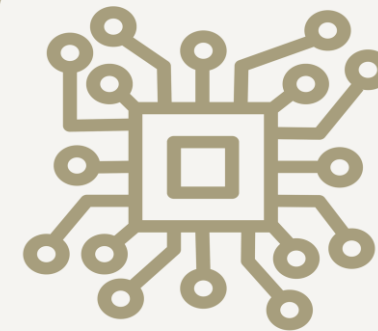
History of CPUs



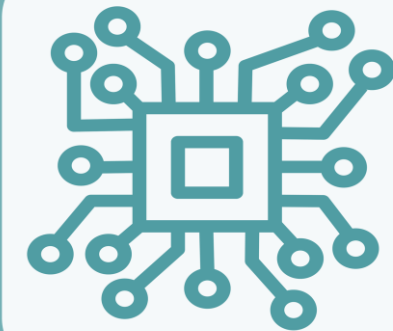
1970-1980s
First CPUs
Few
Megahertz



1990s
100s of
MHz to
over 1 GHz

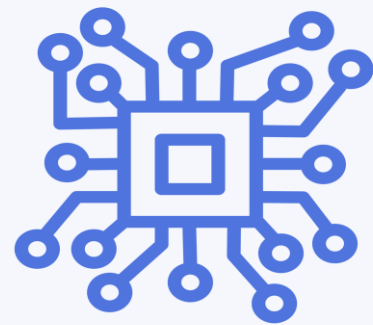


Early 2000s
Top-end
CPUs
over 3 GHz

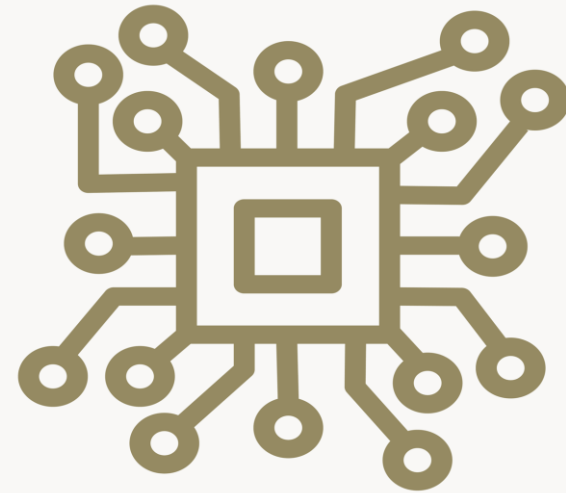


Mid 2000s
3.8 GHz
Thermal
Wall

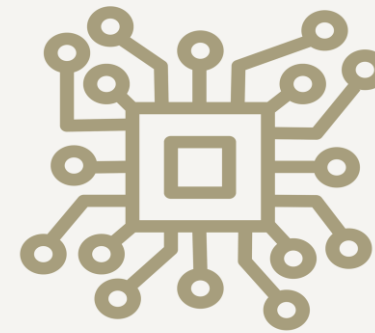
History of CPUs



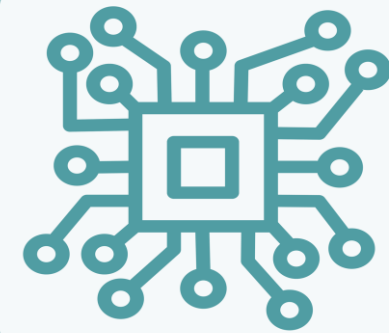
1970-1980s
First CPUs
Few
Megahertz



1990s
100s of MHz
to
over 1 GHz

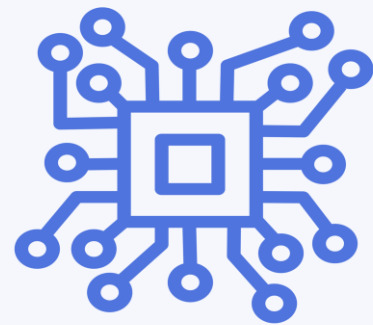


Early 2000s
Top-end
CPUs
over 3 GHz



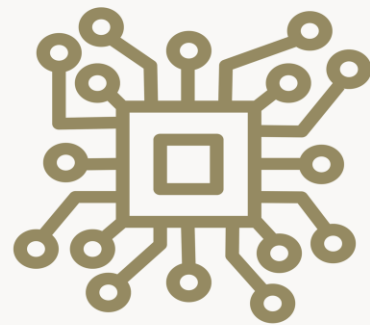
Mid 2000s
3.8 GHz
Thermal
Wall

History of CPUs



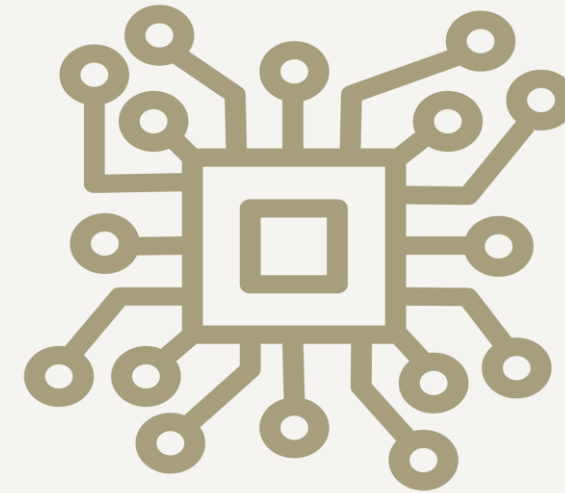
1970-1980s

First CPUs
Few
Megahertz



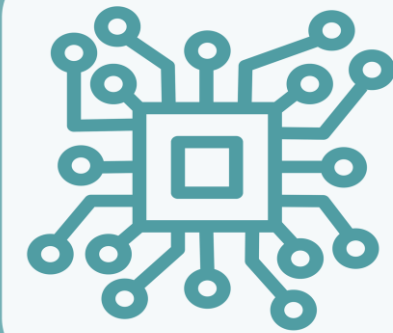
1990s

100s of
MHz to
over 1 GHz



Early 2000s

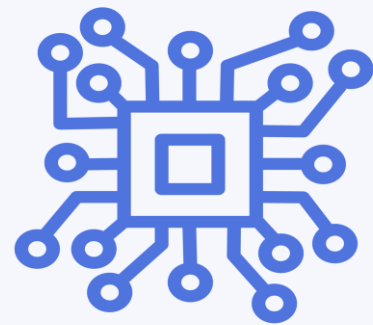
Top-end
CPUs
over 3 GHz



Mid 2000s

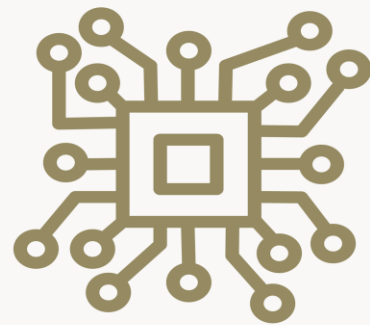
3.8 GHz
Thermal
Wall

History of CPUs



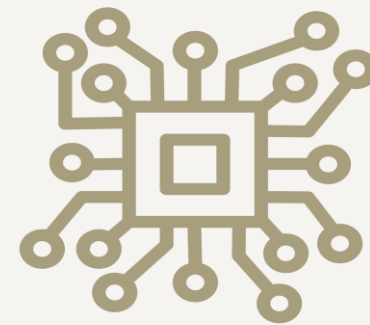
1970-1980s

First CPUs
Few
Megahertz



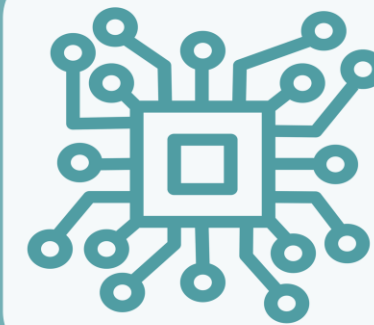
1990s

100s of
MHz to
over 1 GHz



Early 2000s

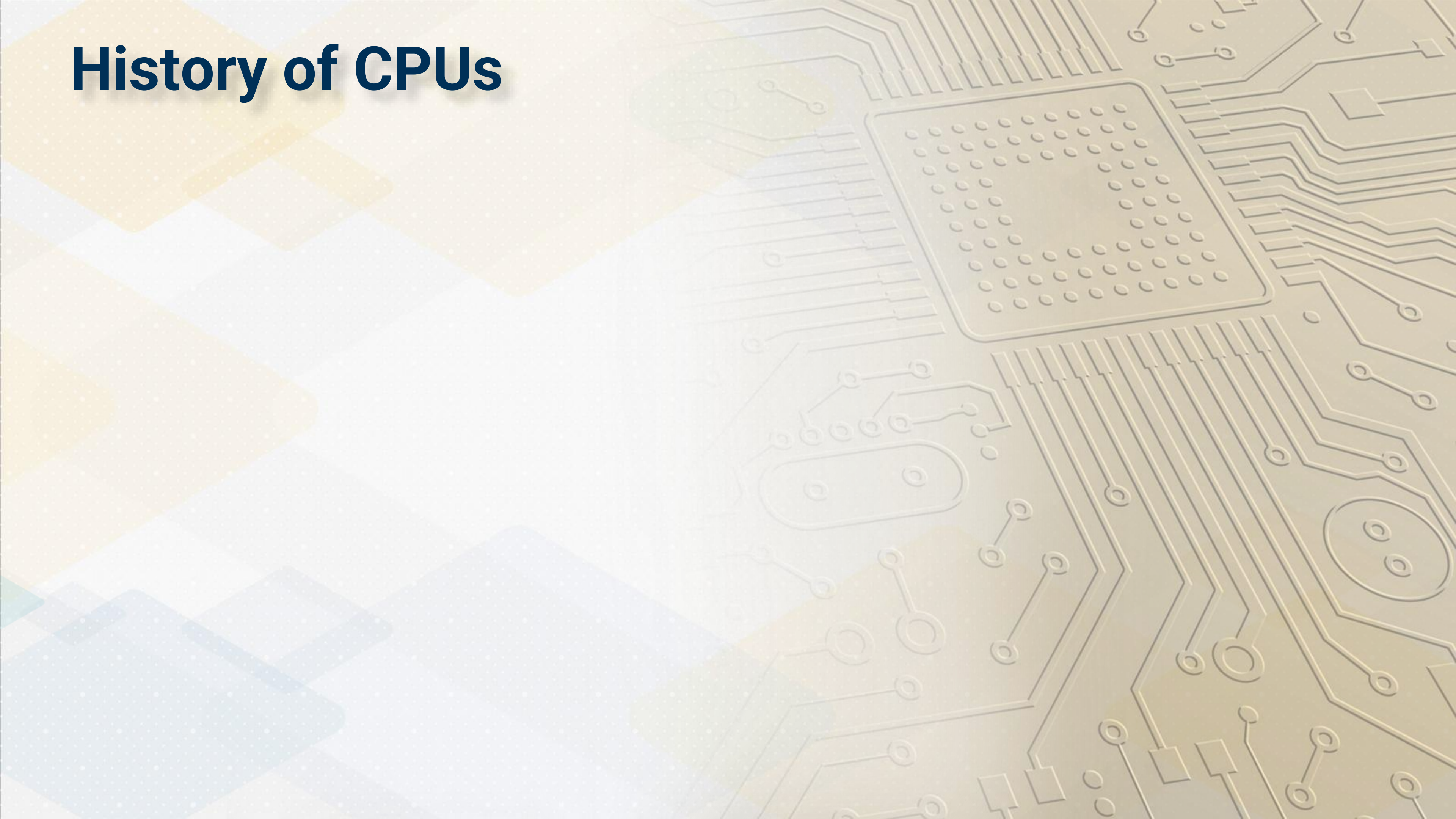
Top-end
CPUs
over 3 GHz



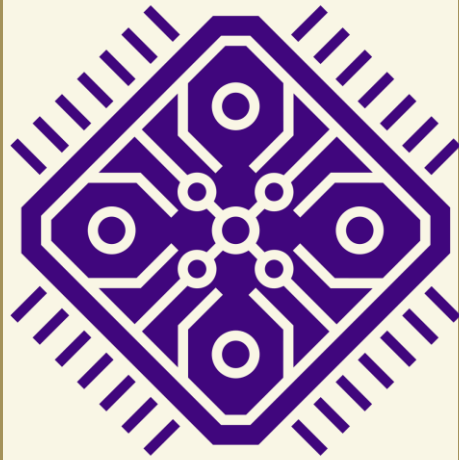
Mid 2000s

3.8 GHz
Thermal
Wall

History of CPUs

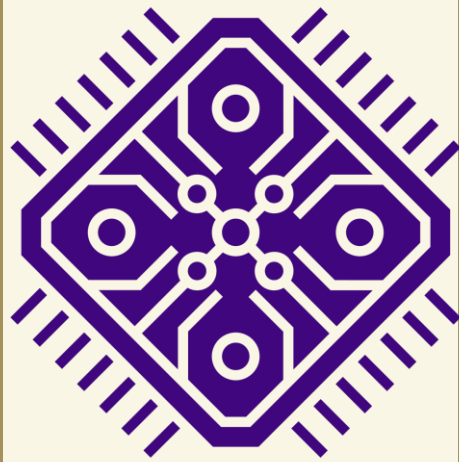


History of CPUs

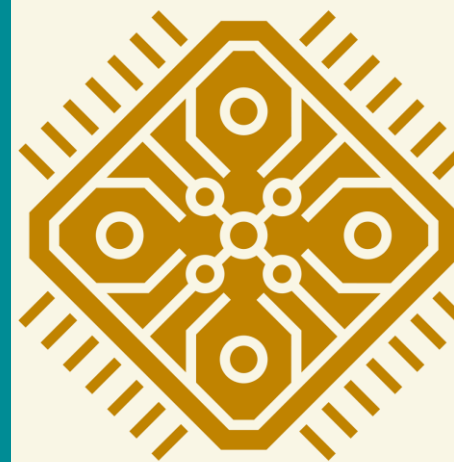


2005
First 2-core
CPUs
Parallel
Processing

History of CPUs

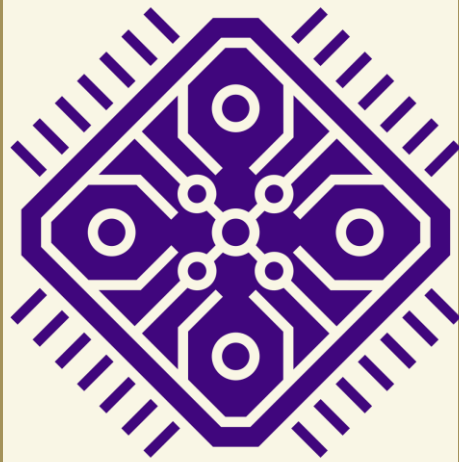


2005
First 2-core
CPUs
Parallel
Processing

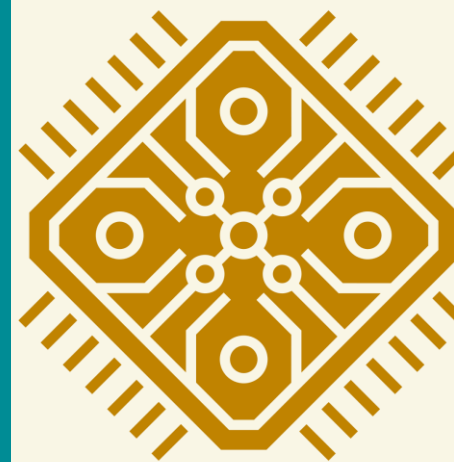


**2006
onwards**
4-core CPUs
AMD
Phenom X4

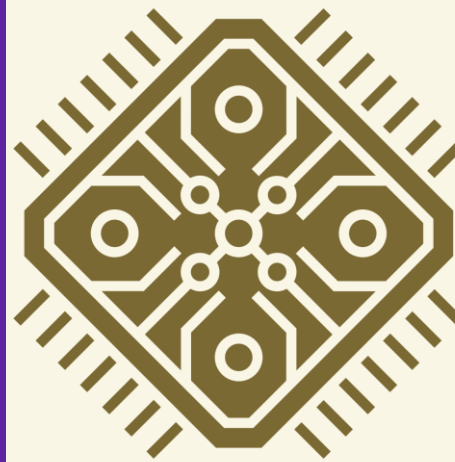
History of CPUs



2005
First 2-core
CPUs
Parallel
Processing

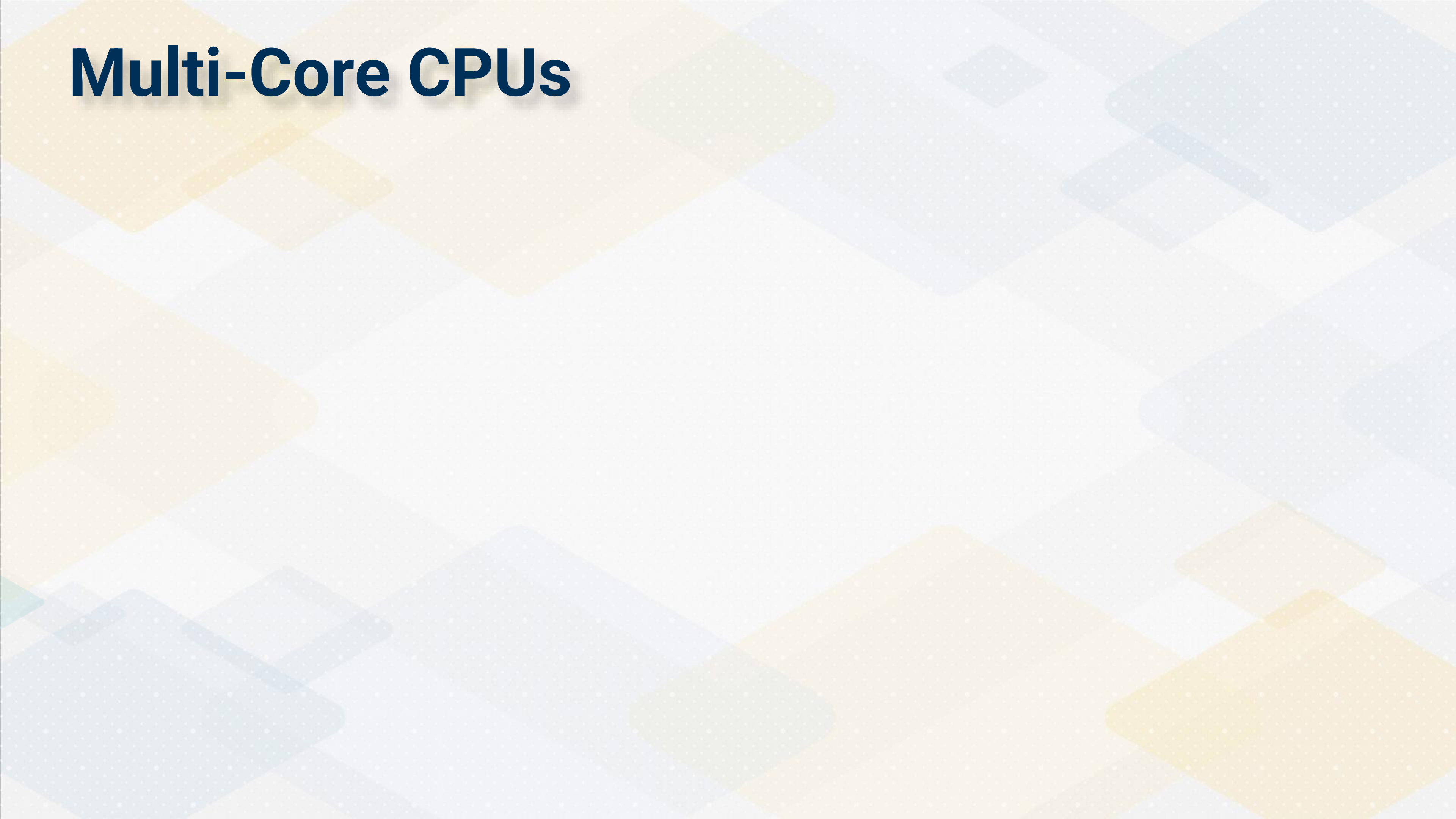


**2006
onwards**
4-core CPUs
AMD
Phenom X4



Today
64-core CPUs
AMD
Threadripper

Multi-Core CPUs

The background of the slide features a white grid pattern. Overlaid on this grid are several semi-transparent squares in two colors: a light yellow and a light blue. These squares are arranged in a staggered, overlapping fashion, creating a sense of depth and a modern, technological aesthetic.

Multi-Core CPUs



Clock Speed

Rhythm of a
Drumbeat

One Beat =
One Instruction

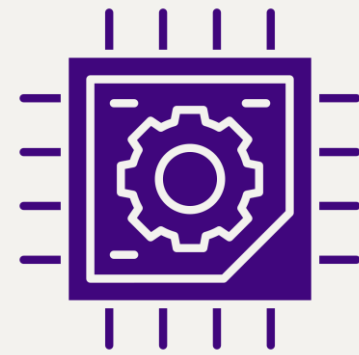
Multi-Core CPUs



Clock Speed

Rhythm of a
Drumbeat

One Beat =
One Instruction



Single-Core CPU

Faster Drumbeat

Causes
Overheating

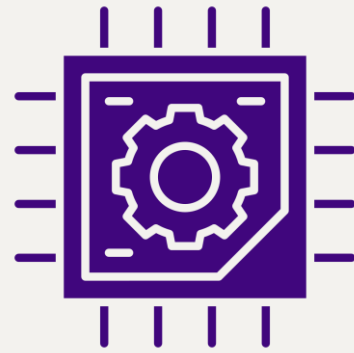
Multi-Core CPUs



Clock Speed

Rhythm of a
Drumbeat

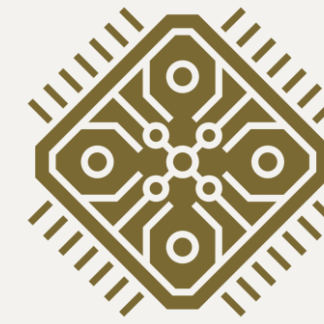
One Beat =
One Instruction



Single-Core CPU

Faster Drumbeat

Causes
Overheating

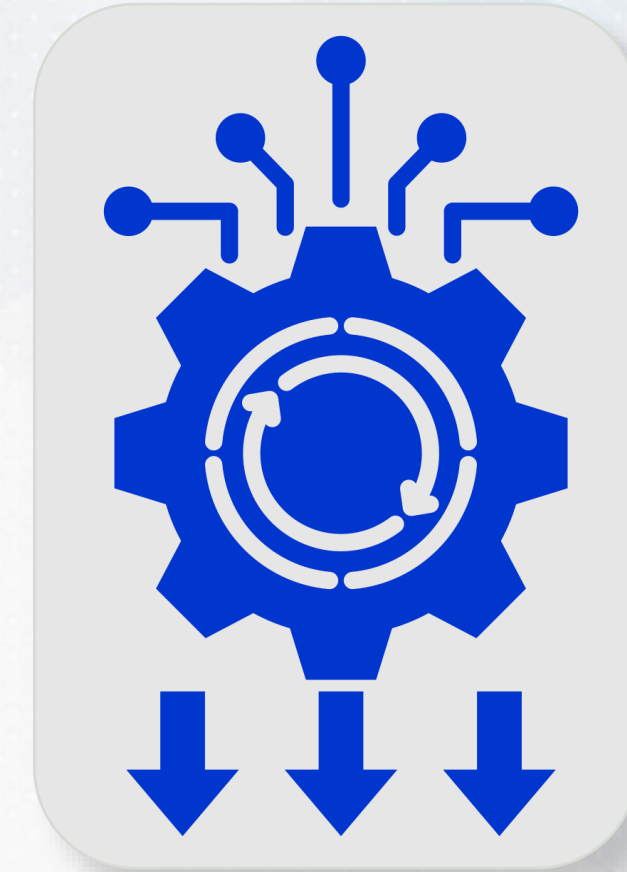


Multi-Core CPU

Multiple
Drummers

Multiple Instructions
at the same time

Threading

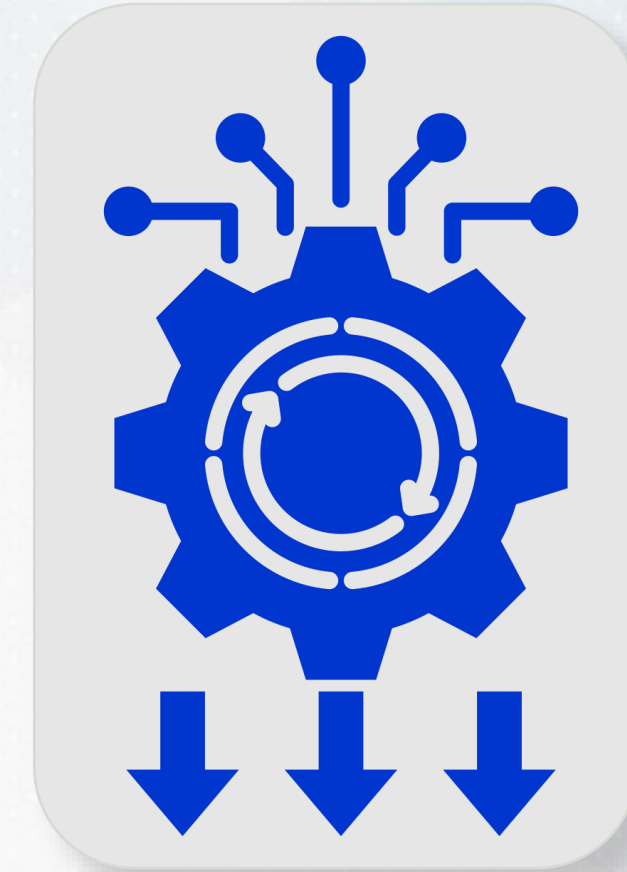


Thread 1
Txn 1 from User 1

Thread 2
Txn 2 from User 2

Threading

Thread: Series of Instructions



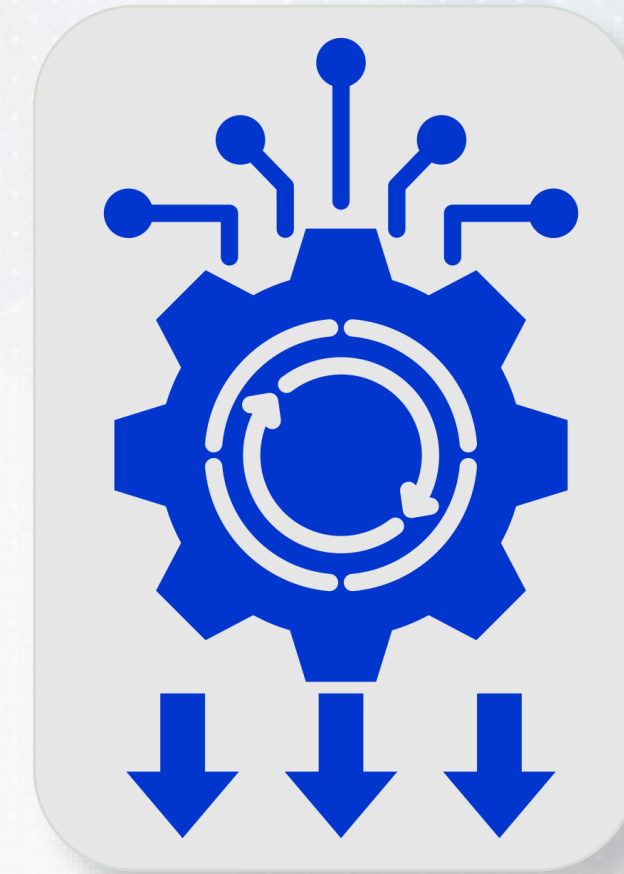
Thread 1
Txn 1 from User 1

Thread 2
Txn 2 from User 2

Threading

Thread: Series of Instructions

Multi-Core CPUs:
Multiple Threads in Parallel



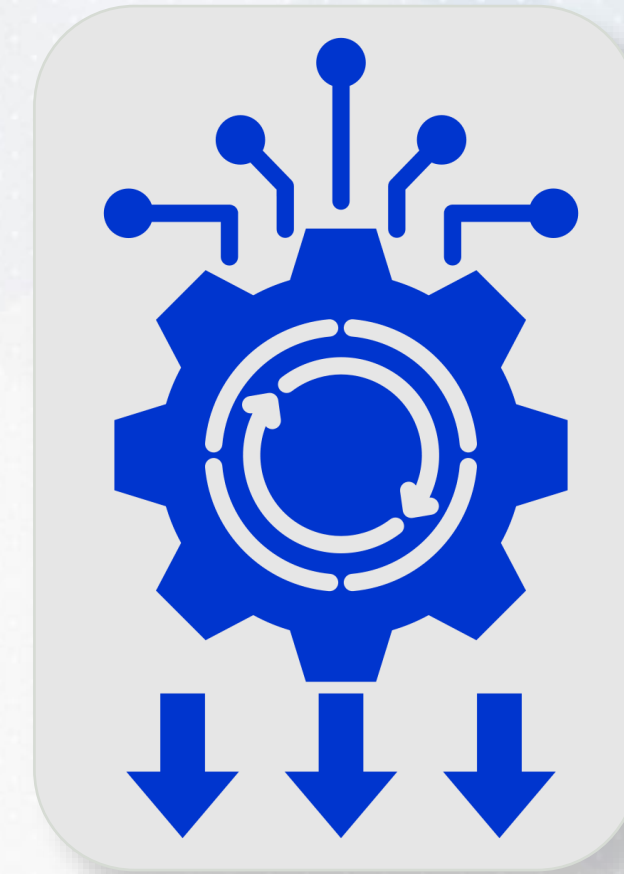
Thread 1
Txn 1 from User 1

Thread 2
Txn 2 from User 2

Threading

Thread: Series of Instructions

Multi-Core CPUs:
Multiple Threads in Parallel



Thread 1
Txn 1 from User 1



Thread 2
Txn 2 from User 2

Multi-Threading Example

Context

A shared bank balance variable updated by multiple threads representing different transactions

```
#include <iostream>
#include <thread>
#include <vector>

int bankBalance = 1000; // Initial bank balance

void performTransactions() {
    for (int i = 0; i < 10000; ++i) {
        bankBalance += 10; // Deposit
        bankBalance -= 10; // Withdrawal
    }
}
```

Multi-Threading Example

```
int main() {  
    std::vector<std::thread> threads;  
    for (int i = 0; i < 8; ++i) {  
        // Initiate transactions without synchronization  
        threads.emplace_back(performTransactions);  
    }  
    for (auto &thread : threads) {  
        thread.join(); // Wait for all threads to finish  
    }  
    std::cout << "Expected balance: 1000\n  
        Actual balance: " << bankBalance  
        << std::endl;  
    return 0;  
}
```

Need for Synchronization

Challenge

Without synchronization, simultaneous transactions lead to inaccurate balance

Run 1

Expected Balance: \$1000

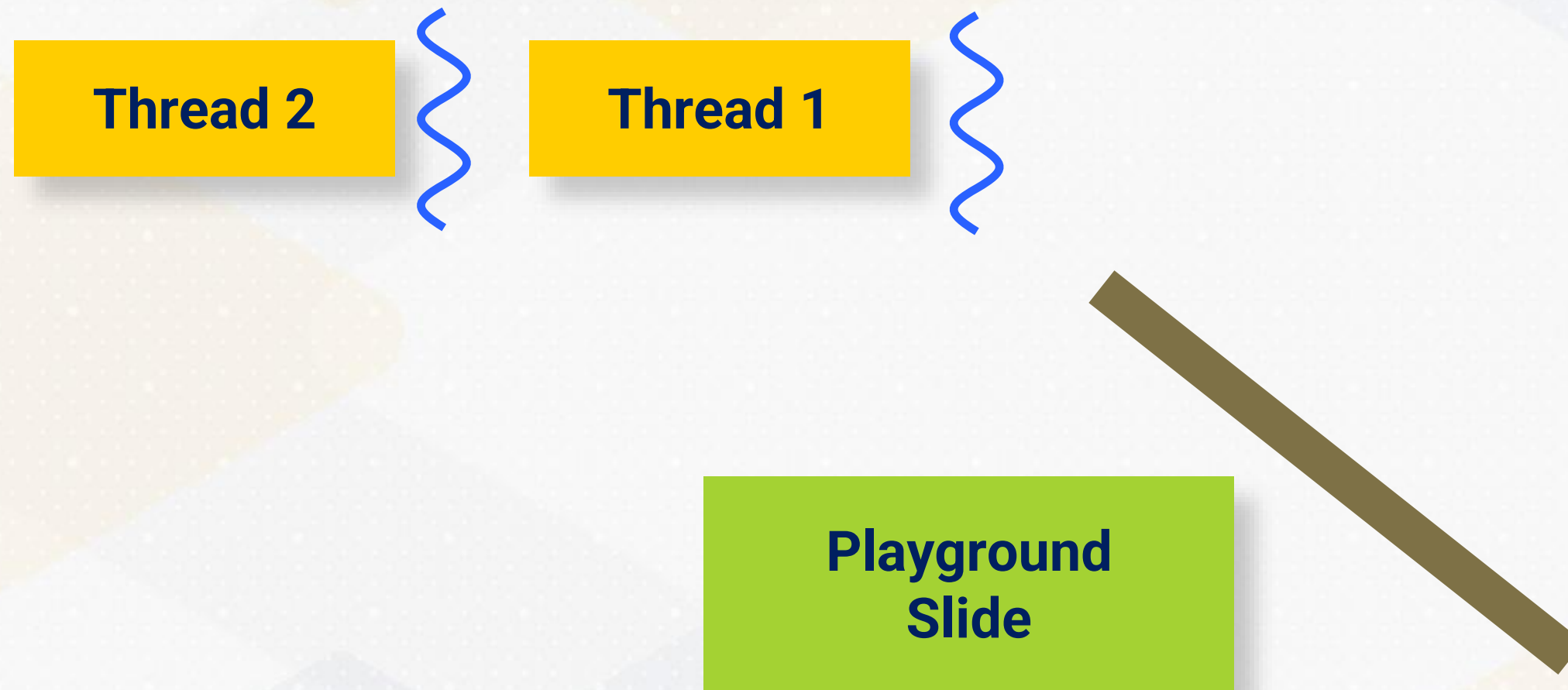
Actual Balance : **\$910**

Run 2

Expected Balance: \$1000

Actual Balance : **\$1020**

Playground Slide Synchronization



Playground Slide Synchronization

Thread 2



Playground
Slide



Thread 1



Playground Slide Synchronization



Assembly Level Explanation

bankBalance += 10;

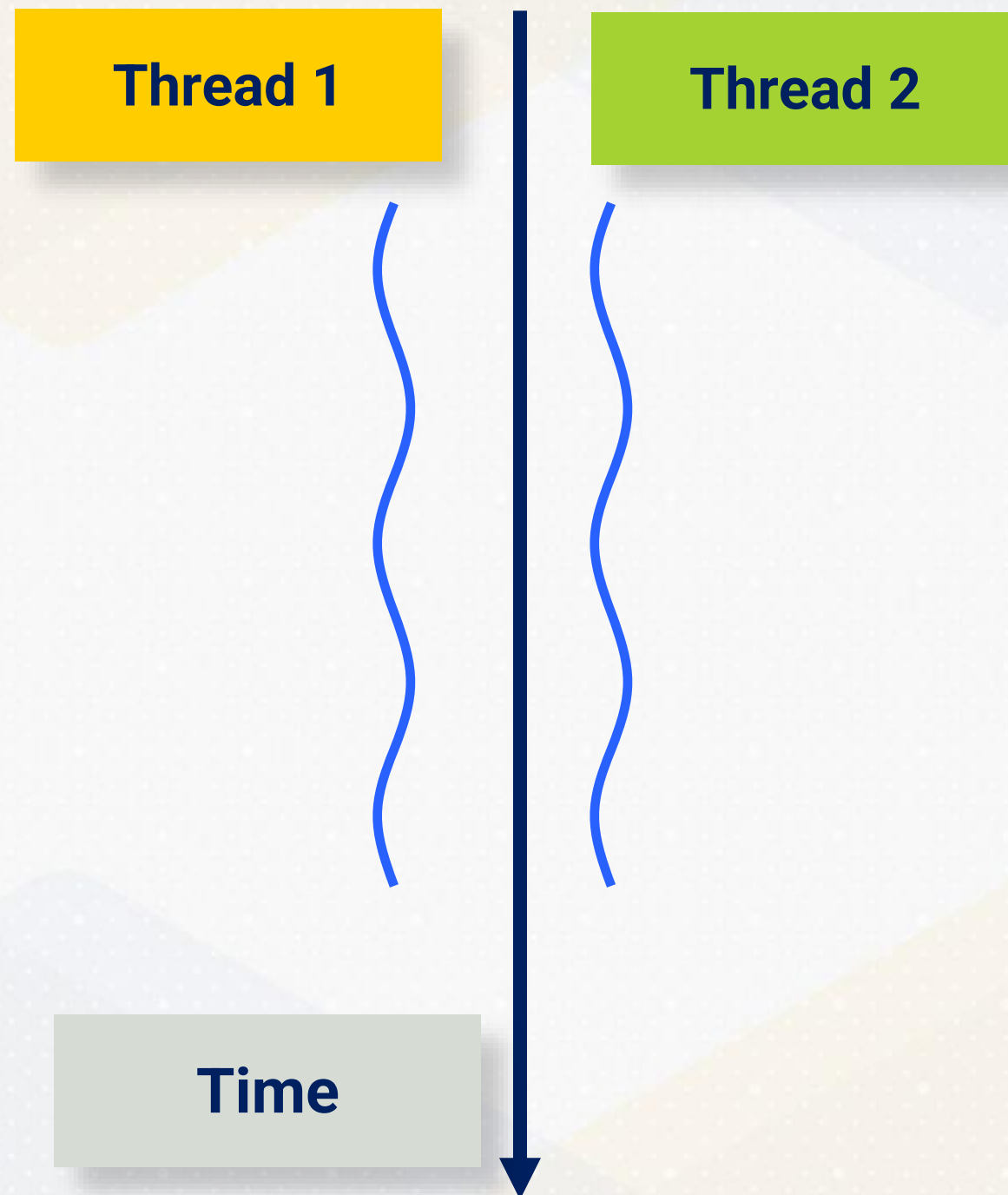


```
// LOAD value of bankBalance from memory to register  
MOV EAX, [bankBalance]
```

```
//ADD 10 to increment the value in the register  
ADD EAX, 10
```

```
//STORE the new value in register back to the memory location of bankBalance  
MOV [bankBalance], EAX
```

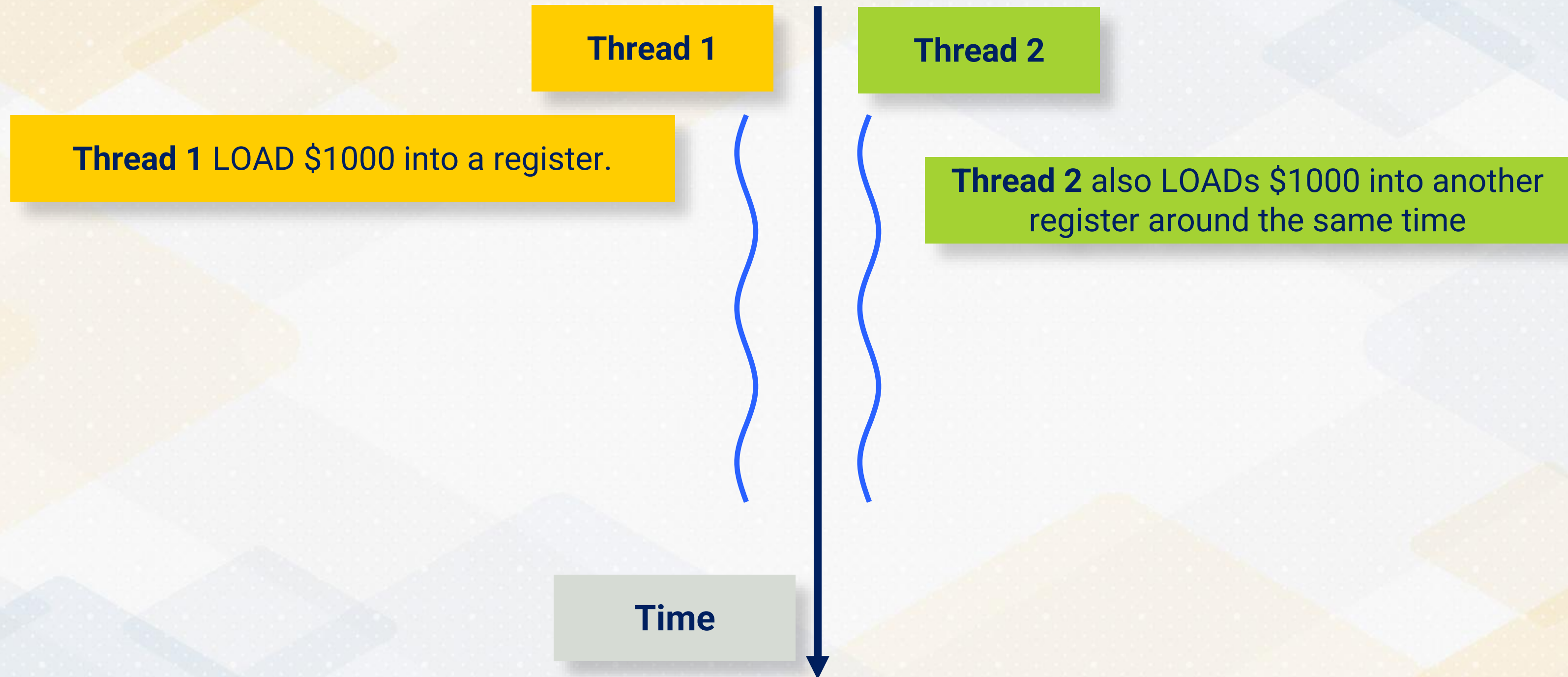
Race Condition



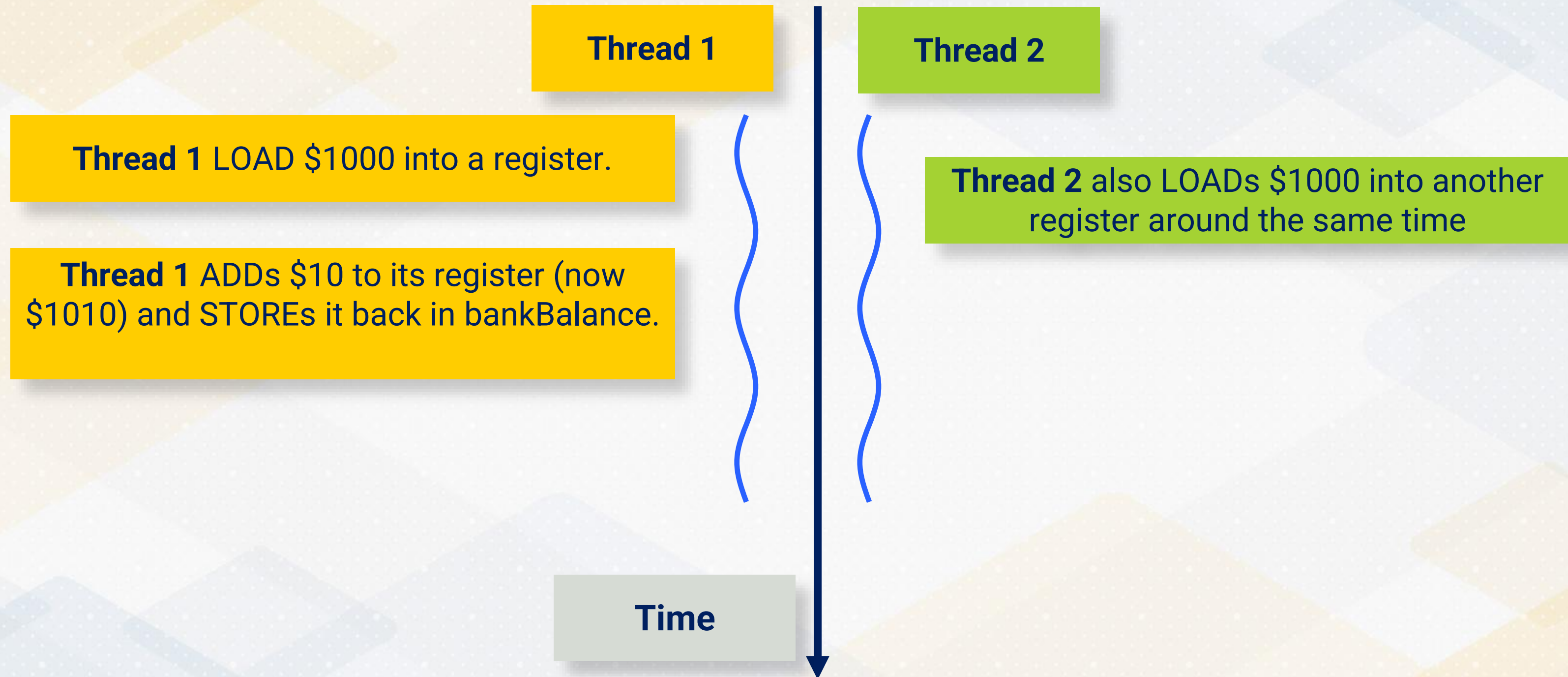
Race Condition



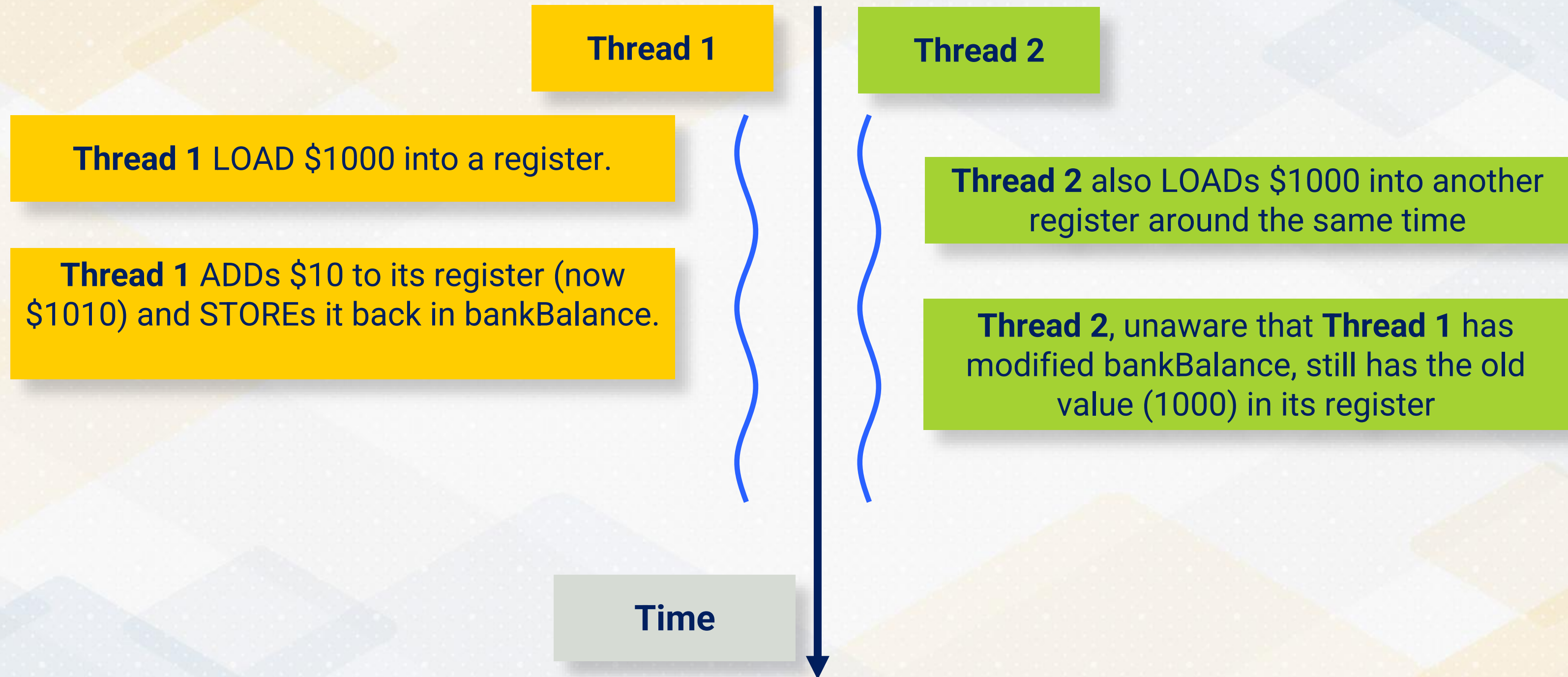
Race Condition



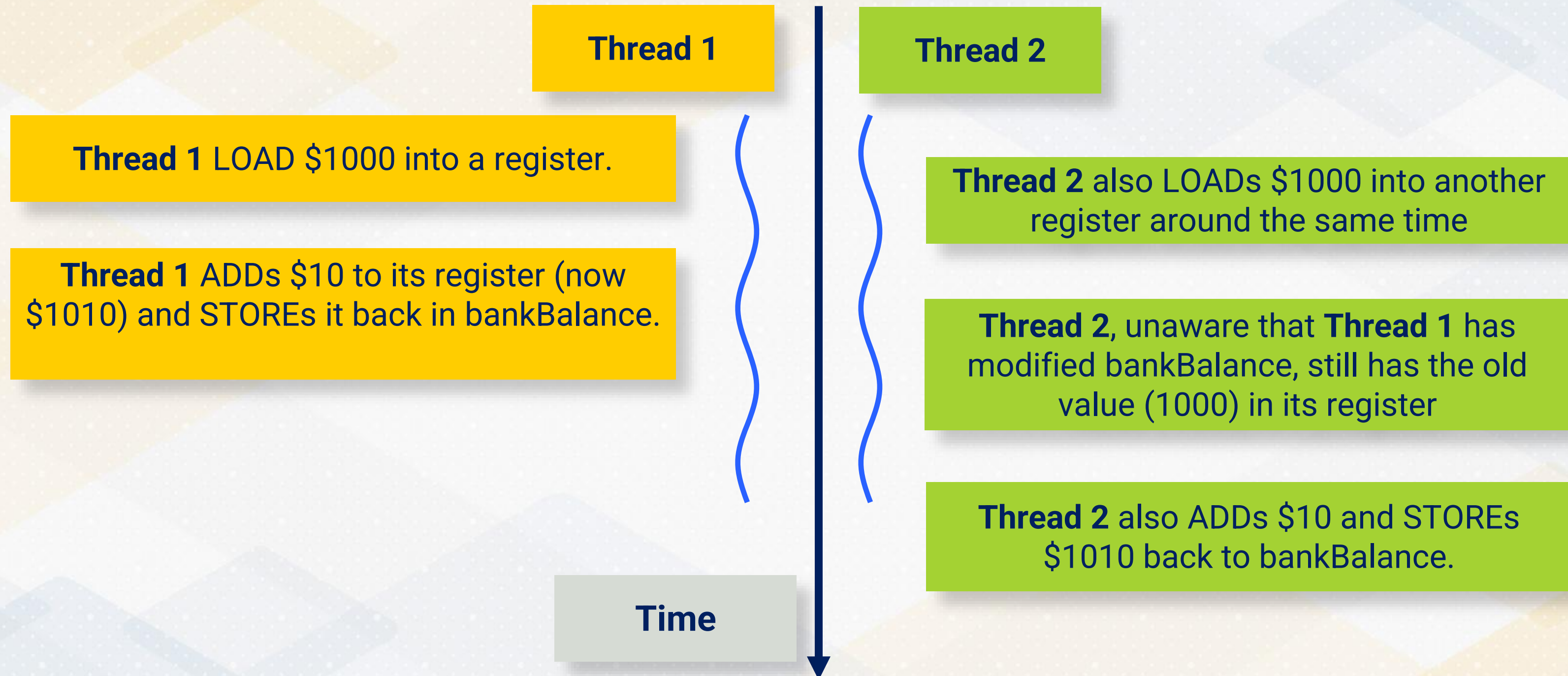
Race Condition



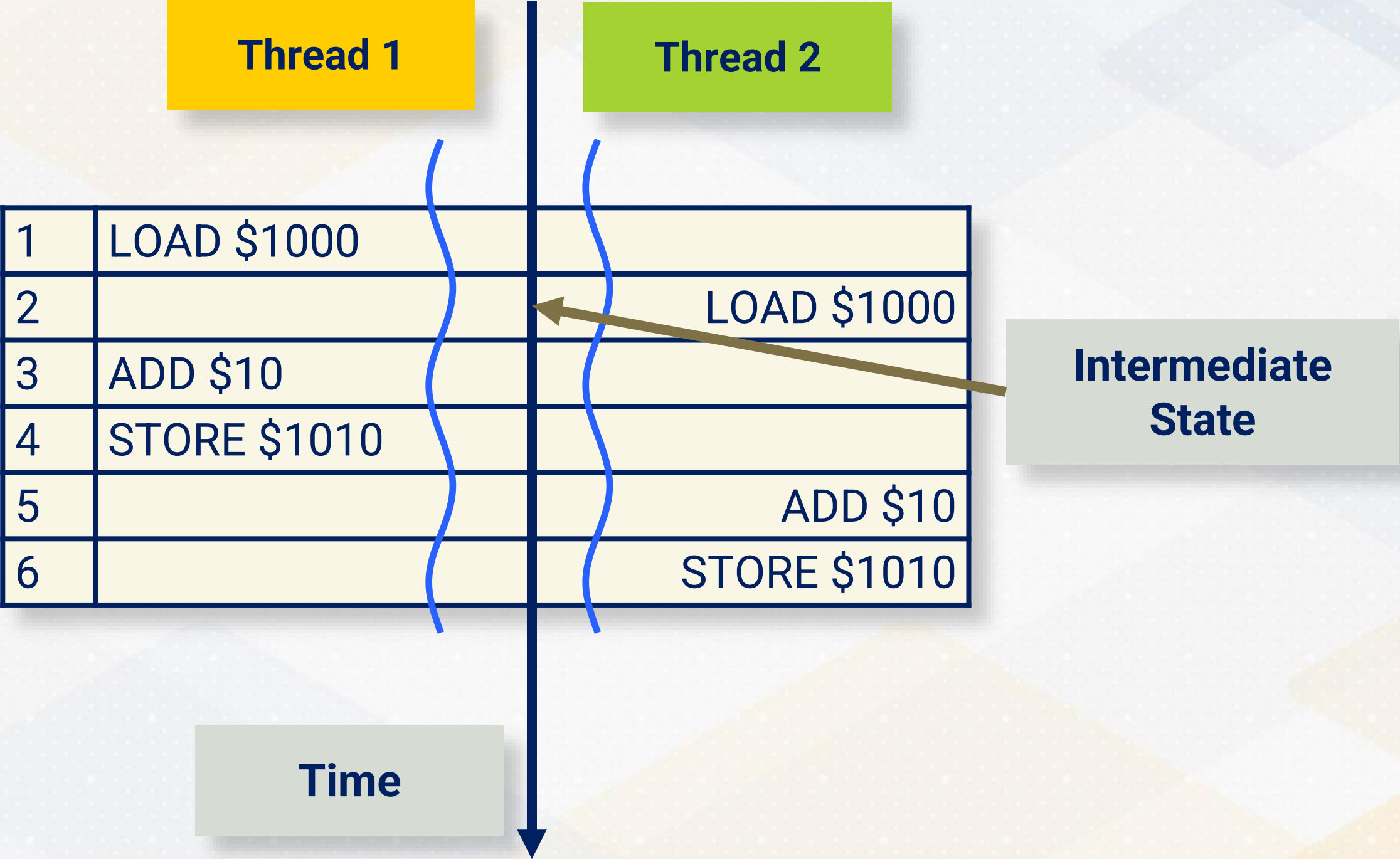
Race Condition



Race Condition



Non-Atomic Load-ADD-STORE Sequence







Multi-Core CPUs



**Multi-Core
CPUs**

**Multi-
Threading
In C++**



Synchronization







Mutual Exclusion



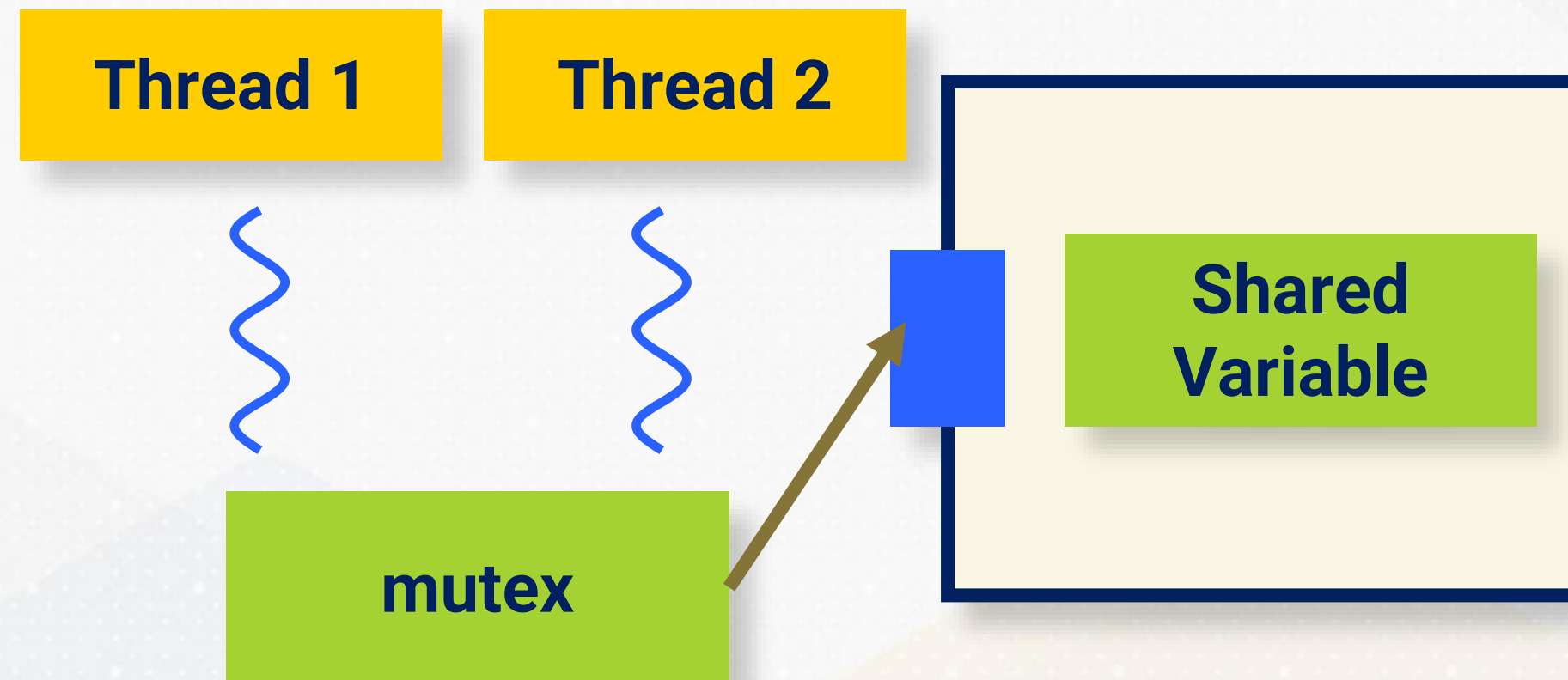
**Mutual
Exclusion**

`std::mutex`

std::mutex (**mut**ual **ex**clusion)

mutex

a door lock to a room with the shared variable



std::mutex (**mut**ual **ex**clusion)



Mutex manual
locking &
unlocking

```
std::mutex bankMutex;  
  
void performTransactions(int account) {  
    for (int i = 0; i < 10000; ++i) {  
        bankMutex.lock(); // Manually lock the mutex  
        bankBalance += 10; // Deposit  
        bankBalance -= 10; // Withdrawal  
        bankMutex.unlock(); // Manually unlock the mutex  
    }  
}
```

std::mutex (**mut**ual **ex**clusion)



Mutex manual
locking &
unlocking

Transactions
processed
atomically

```
std::mutex bankMutex;
```

```
void performTransactions(int account) {  
    for (int i = 0; i < 10000; ++i) {  
        bankMutex.lock(); // Manually lock the mutex  
        bankBalance += 10; // Deposit  
        bankBalance -= 10; // Withdrawal  
        bankMutex.unlock(); // Manually unlock the mutex  
    }  
}
```

std::mutex (**mut**ual **ex**clusion)



Mutex manual
locking &
unlocking

Transactions
processed
atomically

Prevents data
corruption

```
std::mutex bankMutex;
```

```
void performTransactions(int account) {  
    for (int i = 0; i < 10000; ++i) {  
        bankMutex.lock(); // Manually lock the mutex  
        bankBalance += 10; // Deposit  
        bankBalance -= 10; // Withdrawal  
        bankMutex.unlock(); // Manually unlock the mutex  
    }  
}
```

Critical Section



- Operations between lock and unlock form critical section
- Only one thread enters the section at a time

```
bankMutex.lock(); // Manually lock the mutex  
// CRITICAL SECTION STARTS  
bankBalance += 10; // Deposit  
bankBalance -= 10; // Withdrawal  
// CRITICAL SECTION ENDS  
bankMutex.unlock(); // Manually unlock the mutex
```

Mutex Operations at Assembly Level

; Locking the Mutex

retry:

; EAX is set to the expected old value (unlocked = 0)

MOV EAX, 0

; EBX is set to the new value to store if comparison is successful (locked = 1)

MOV EBX, 1

; Atomically compare [mutex] to 0, if equal replace [mutex] with 1

LOCK CMPXCHG [mutex], EBX;

; Test if the original value of [mutex] (now in EBX) was 1

TEST EBX, EBX;

; If the mutex was already locked (EBX was 1), jump to retry

JNZ retry

Mutex Operations at Assembly Level

; Critical section to update bankBalance

MOV EAX, [bankBalance] ; Load bank balance

ADD EAX, 10 ; Modify bank balance

MOV [bankBalance], EAX ; Store bank balance

; Unlocking the Mutex

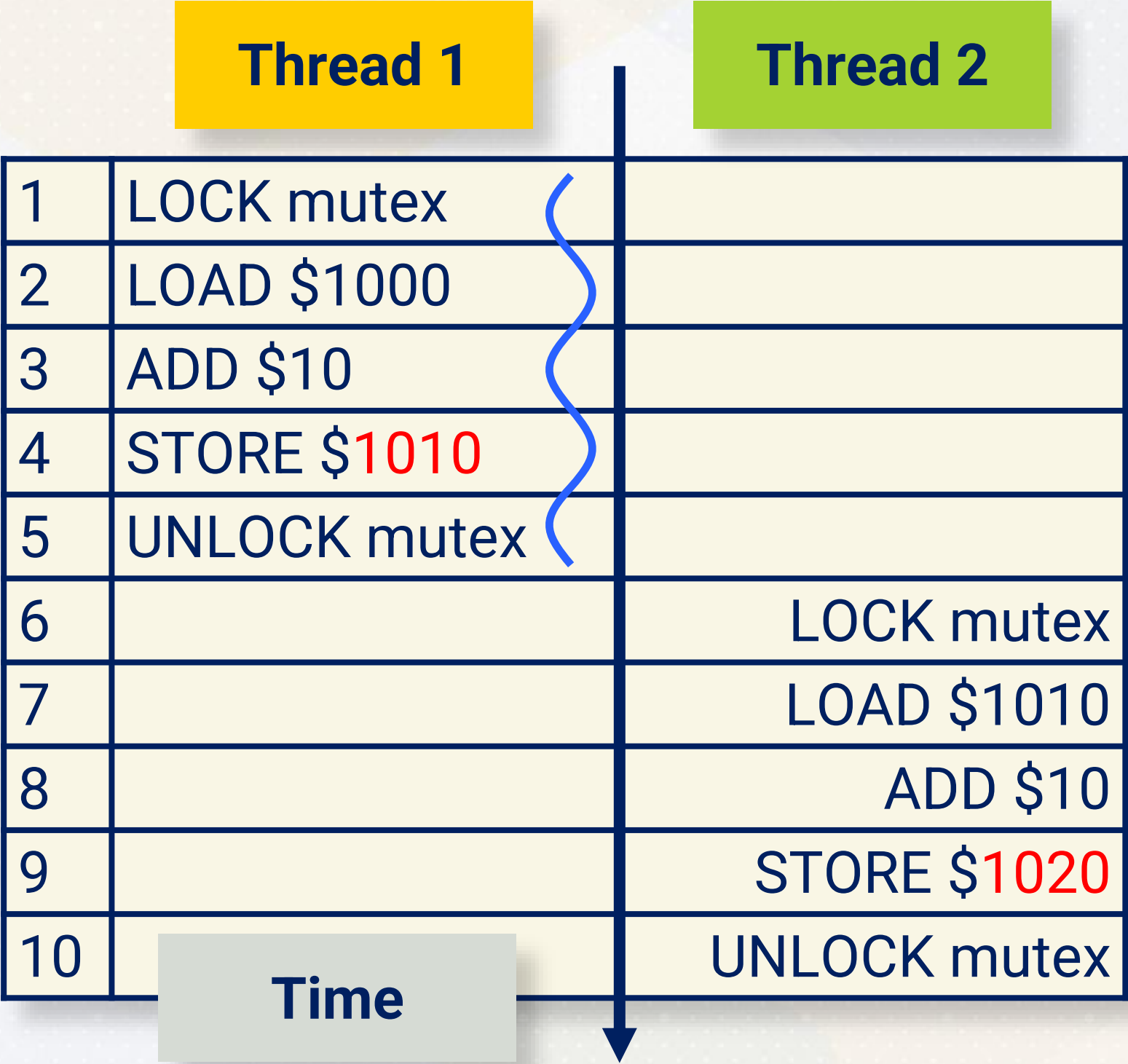
MOV EBX, 0 ; Set EBX to 0, which represents the unlocked state

MOV [mutex], EBX ; Store 0 into the mutex, effectively unlocking it

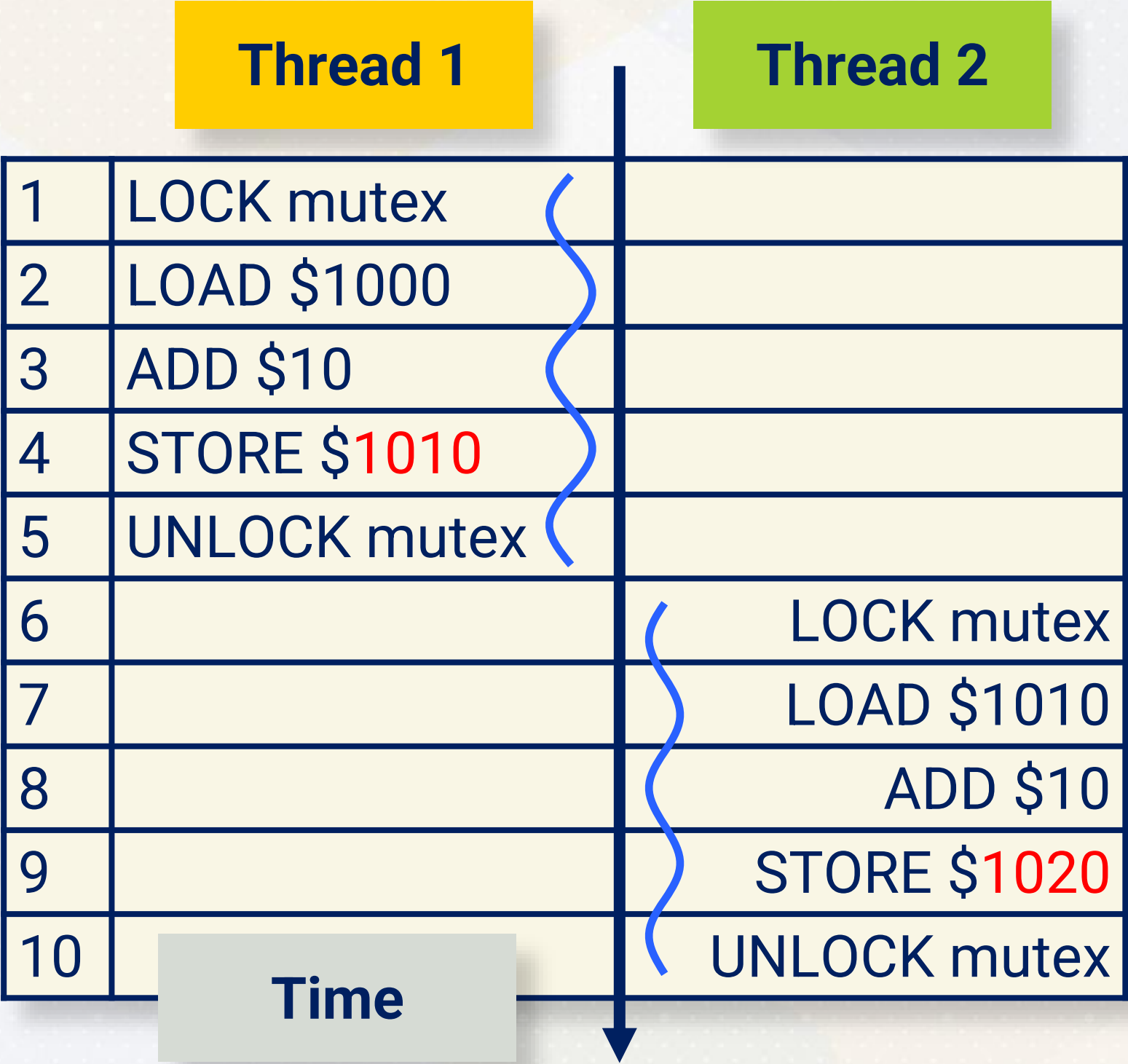
Mutex Prevents Data Race

	Thread 1	Thread 2
1	LOCK mutex	
2	LOAD \$1000	
3	ADD \$10	
4	STORE \$1010	
5	UNLOCK mutex	
6		LOCK mutex
7		LOAD \$1010
8		ADD \$10
9		STORE \$1020
10		UNLOCK mutex
	Time	

Mutex Prevents Data Race



Mutex Prevents Data Race

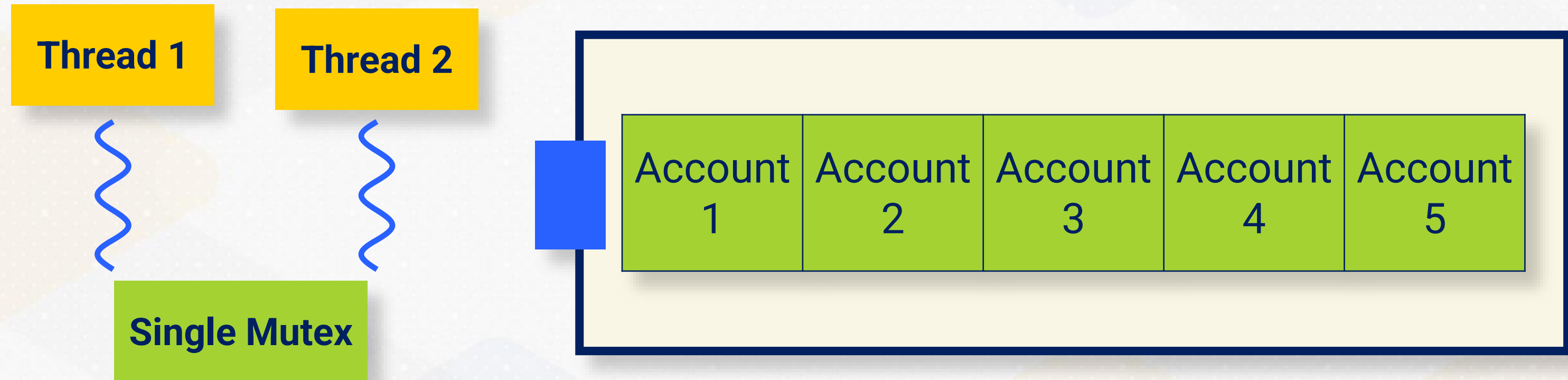


Multiple Bank Accounts

Managing transactions across 5 bank accounts in a multi-threaded application

```
std::mutex bankMutex;  
// Initial balances of five bank accounts  
int bankAccounts[5] = {1000, 2000, 3000, 4000, 5000};  
  
void performTransactions(int account) {  
    for (int i = 0; i < 10000; ++i) {  
        bankMutex.lock();          // Single mutex for all five accounts  
        bankAccounts[account] += 10; // Deposit  
        bankAccounts[account] -= 10; // Withdrawal  
        bankMutex.unlock();  
    }  
}
```

Multiple Bank Accounts







Mutual Exclusion



**Mutual
Exclusion**

`std::mutex`



Fine-Grained Locking







Fine-Grained Locking

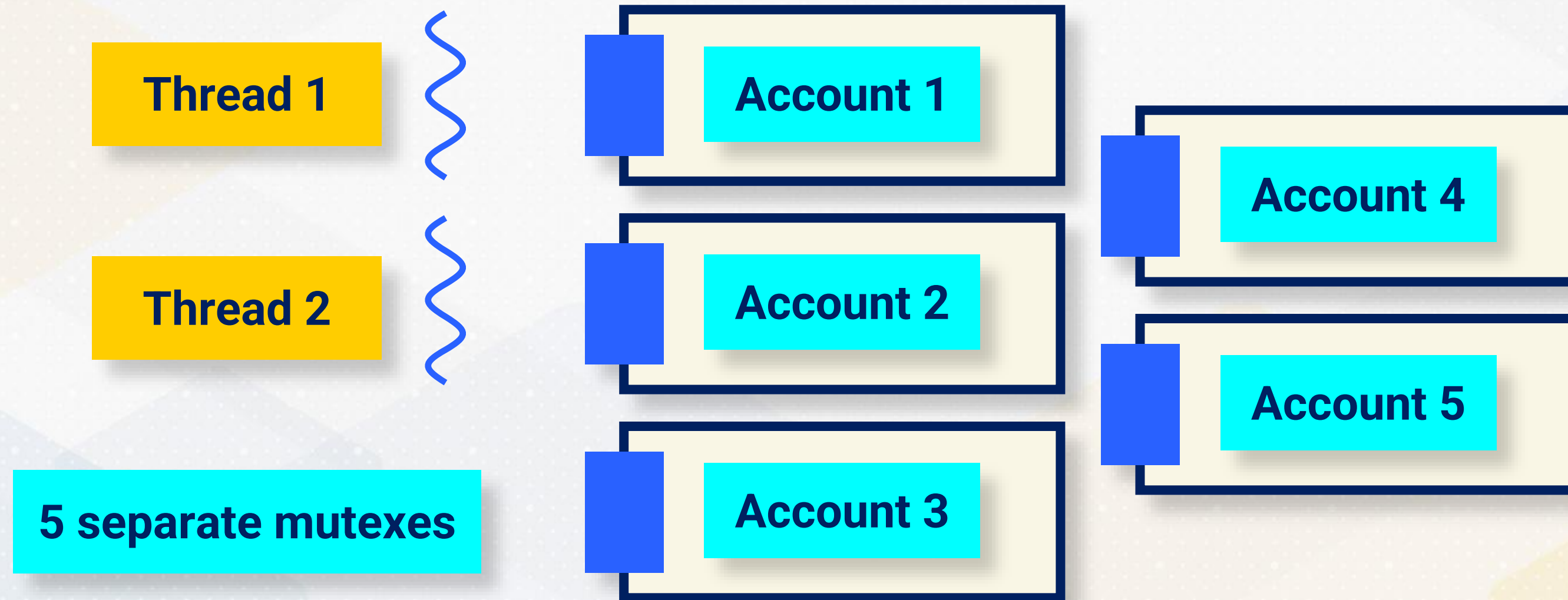


**Fine-Grained
Locking**

**std::
lock_guard**

Fine-Grained Locking

Use a separate mutex for each bank account to improve concurrency



Fine-Grained Locking

Non-conflicting transactions can run in parallel

```
std::mutex bankAccountMutexes[5]; // A mutex for each bank account
int bankAccounts[5] = {1000, 2000, 3000, 4000, 5000}; // Initial balances

void performTransactions(int account) {
    for (int i = 0; i < 10000; ++i) {
        bankAccountMutexes[account].lock(); // Lock only the mutex for specified account
        bankAccounts[account] += 10; // Deposit
        bankAccounts[account] -= 10; // Withdrawal
        bankAccountMutexes[account].unlock(); // Unlock the mutex for the specified account
    }
}
```

Manual Locking and Unlocking

Manual mutex locking and unlocking comes with risk

```
void performTransactions(int account) {  
    for (int i = 0; i < 10000; ++i) {  
        bankAccountMutexes[account].lock(); // Lock only the mutex for specified account  
        bankAccounts[account] += 10; // Deposit  
        bankAccounts[account] -= 10; // Withdrawal  
        bankAccountMutexes[account].unlock(); // Unlock the mutex for the specified account  
    }  
}
```

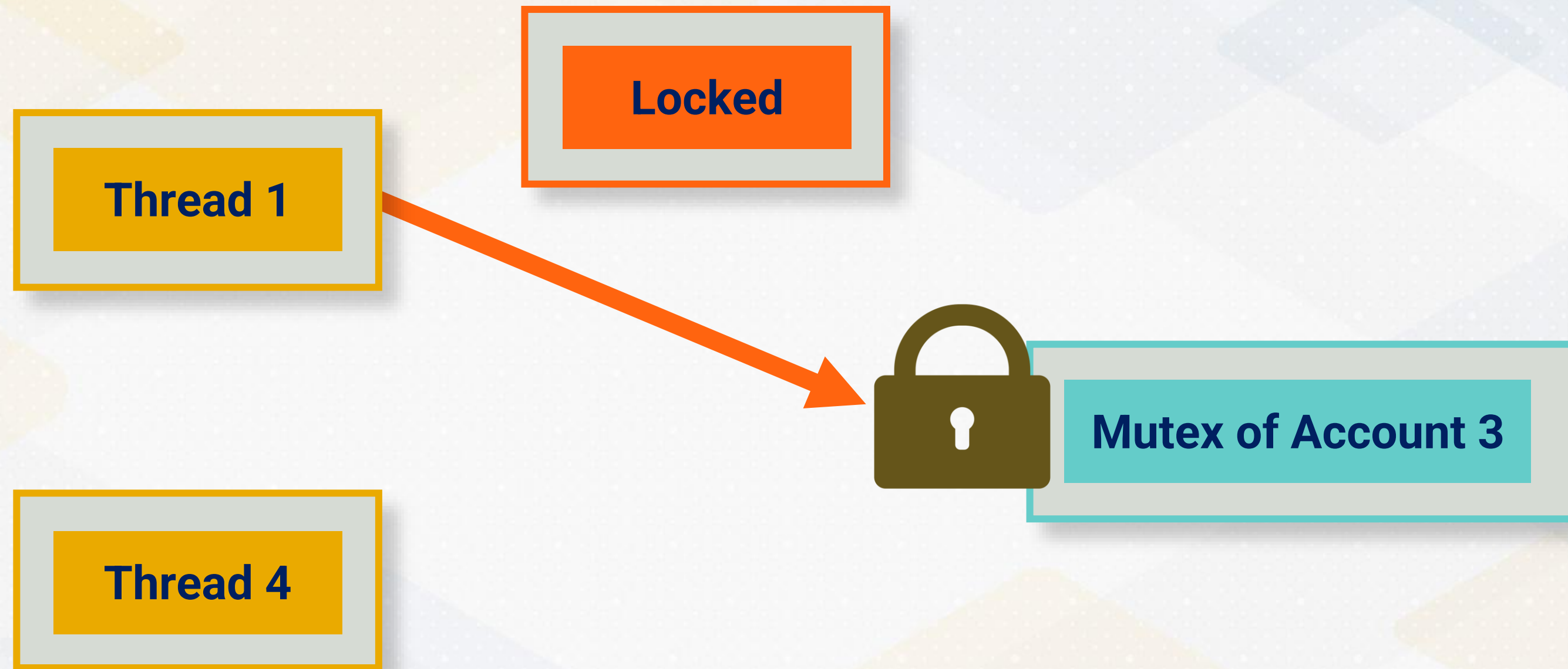
Thread Starvation

Thread 1

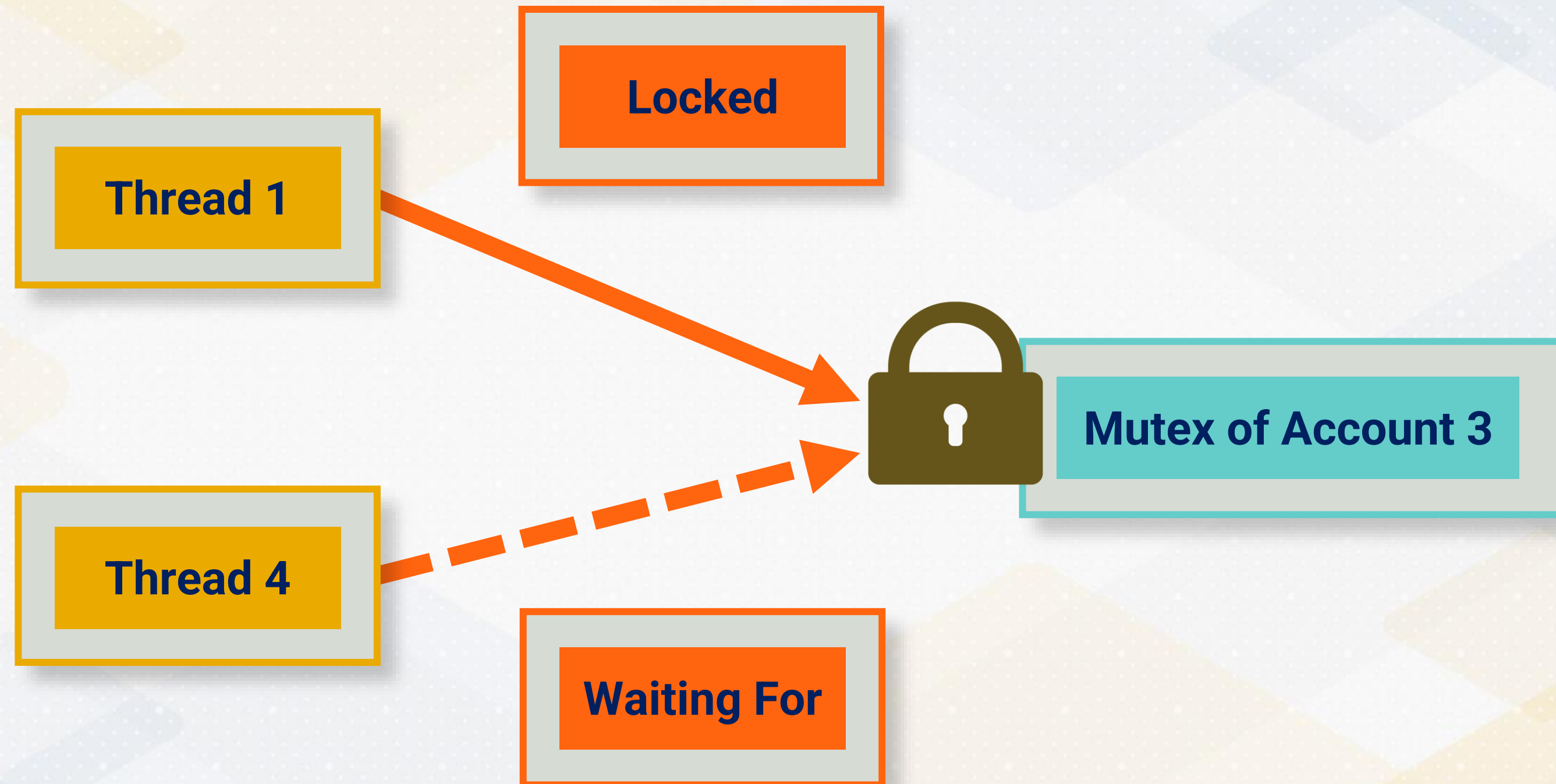
Thread 4

Mutex of Account 3

Thread Starvation



Thread Starvation



std::lock_guard

A solution to avoid forgetting to **unlock a mutex** is to use **std::lock_guard**

```
void performTransactions(int account) {  
    for (int i = 0; i < 10000; ++i) {  
        // Automatically locks  
        std::lock_guard<std::mutex> lock(bankAccountMutexes[account]);  
        bankAccounts[account] += 10; // Deposit  
        bankAccounts[account] -= 10; // Withdrawal  
        // Mutex automatically unlocked when lock goes out of scope  
    }  
}
```

std::lock_guard automatically manages locking and unlocking

RAII Principle in C++

std::lock_guard another example of RAII

```
std::mutex myMutex;  
std::lock_guard<std::mutex> lock(myMutex); // Object created here
```

Resource mutex

Object instance of std::lock_guard

RAII Principle in C++

Automate resource management by
tying resource allocation to object lifespan

RAII Object	Resource Managed	Acquisition	Release
<code>std::lock_guard</code>	Mutex	Locks the mutex upon creation	Automatically releases the lock when object is destroyed.
<code>std::unique_ptr</code>	Dynamic memory	Allocates memory and takes ownership	Automatically deallocates memory
<code>std::fstream</code>	File handle	Opens a file and acquires the file handle	Closes the file and releases the file handle





Fine-Grained Locking



**Fine-Grained
Locking**

**std::
lock_guard**

CMYK-Based Tertiaries



IMPACT
PURPLE
(dark purple)
PMS: 267 C
RGB: 95, 36, 159
HEX: #5F249F
CMYK: 81, 99, 0, 0



ELECTRIC
BLUE
(light green-blue)
PMS: 325 C
RGB: 100 204 201
HEX: #64CCC9
CMYK: 54, 0, 20, 0



RAT CAP
(warm yellow)
PMS: 116 C
RGB: 255, 205, 0
HEX: #FFCD00
CMYK: 0, 10, 98, 0



BOLD BLUE
(medium blue)
PMS: 7455 C
RGB: 58, 93, 174
HEX: #3A5DAE
CMYK: 86, 66, 0, 0



CANOPY LIME
(light warm green)
PMS: 2299 C
RGB: 164, 210, 51
HEX: #A4D233
CMYK: 38, 0, 94, 0



NEW HORIZON
(orange-red)
PMS: 7417 C
RGB: 224, 79, 57
HEX: #E04F39
CMYK: 0, 82, 82, 0



OLYMPIC
TEAL
(medium green-blue)
PMS: 321 C
RGB: 0, 140, 149
HEX: #008C95
CMYK: 100, 0, 37, 10

Accessible Tech Gold

Use accessible color combinations to enhance legibility and ensure that Georgia Tech's content is available to a diverse audience. **Text in Tech Gold on a white background is not accessible.** When using Tech Gold for text on a website or other digital media, substitute Tech Medium Gold for large text and Tech Dark Gold for small bold text.



TECH MEDIUM GOLD
HEX: #A4925A
RGB: 164, 146, 90
For use in large headline text in any digital media.



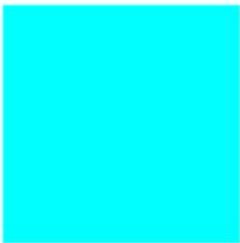
TECH DARK GOLD
HEX: #857437
RGB: 133, 116, 55
For use in small bold text in any digital media.

RGB-Based Tertiaries ("Brights")

The tertiary palette also includes a separate suite of brighter, web-only colors. These color standards maximize visibility and contrast on screens and should not be applied to print projects.



BRIGHT
PURPLE
(violet)
RGB: 120, 0, 255
HEX: #7800FF



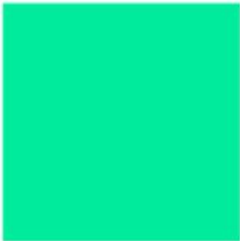
BRIGHT
ELECTRIC
(light teal blue)
RGB: 0, 255, 255
HEX: #00FFFF



BRIGHT BUZZ
(warm yellow)
RGB: 255, 204, 0
HEX: #FFCC00



BRIGHT BLUE
(medium blue)
RGB: 41, 97, 255
HEX: #2961FF



BRIGHT
CANOPY
(cool green)
RGB: 0, 236, 156
HEX: #00EC9C



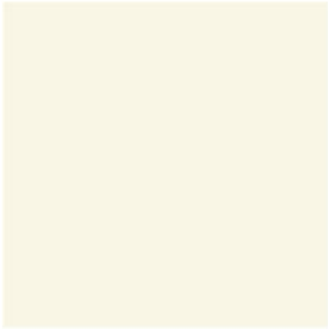
BRIGHT
HORIZON
(red-orange)
RGB: 255, 100, 15
HEX: #FF640F



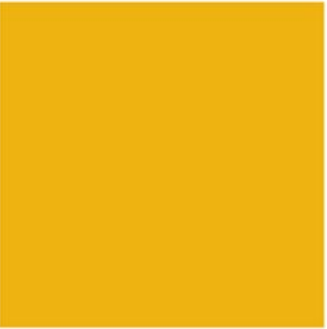
GRAY MATTER
(neutral dark gray)
PMS: 425
HEX: #54585A
RGB: 84, 88, 90
HSL: 200, 3, 34
HSB: 200, 7, 35
CMYK: 48, 29, 26, 76



PI MILE
(light warm gray)
PMS: 427
HEX: #D6DBD4
RGB: 214, 219, 212
HSL: 103, 9, 85
HSB: 103, 3, 86
CMYK: 9, 4, 10, 8



DIPLOMA
(light ivory yellow)
PMS: 7499
HEX: #F9F6E5
RGB: 249, 246, 229
HSL: 51, 63, 94
HSB: 51, 8, 98
CMYK: 1, 1, 10, 1



BUZZ GOLD
PMS: 124
HEX: #EAAA00
RGB: 234, 170, 0
HSL: 44, 100, 46
HSB: 44, 100, 92
CMYK: 0, 30, 100, 0



Database Configuration







Database Configuration



**Database
Configuration**

**Configuration
Tuning**

Database Configuration

Parameter	Value
PAGE_SIZE	1 KB
MAX_SLOTS	128 slots per page
MAX_PAGES_IN_MEMORY	10 pages
...	...

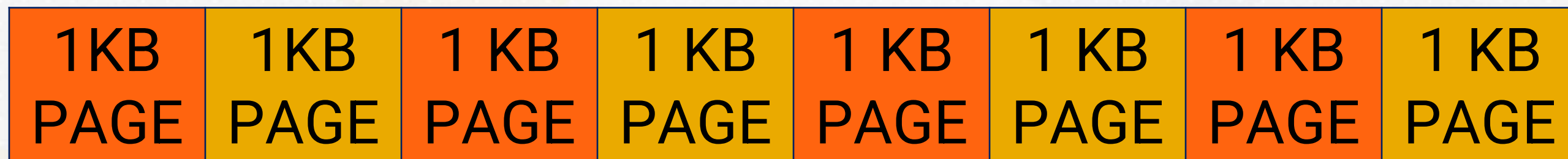
PAGE_SIZE

8KB BUFFER POOL

1KB PAGE	1KB PAGE	1 KB PAGE	1 KB PAGE	1 KB PAGE	1 KB PAGE	1 KB PAGE	1 KB PAGE
-------------	-------------	--------------	--------------	--------------	--------------	--------------	--------------

PAGE_SIZE

8KB BUFFER POOL



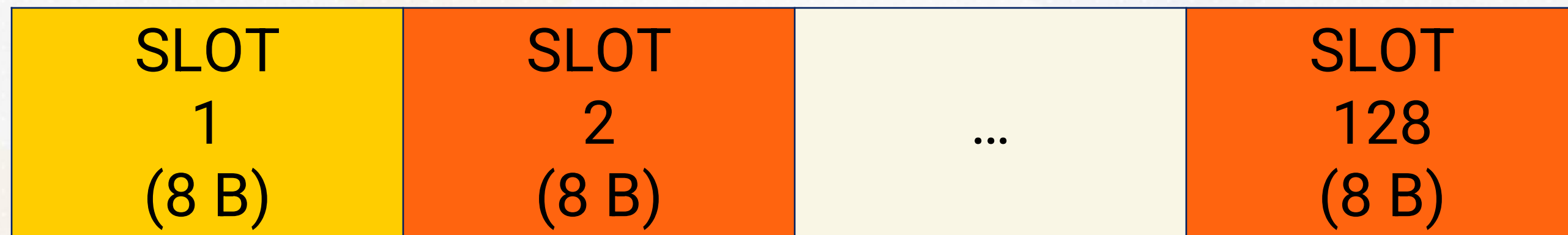
MAX_SLOTS

1KB SLOTTED PAGE

SLOT 1 (8 B)	SLOT 2 (8 B)	...	SLOT 128 (8 B)
--------------------	--------------------	-----	----------------------

MAX_SLOTS

1KB SLOTTED PAGE



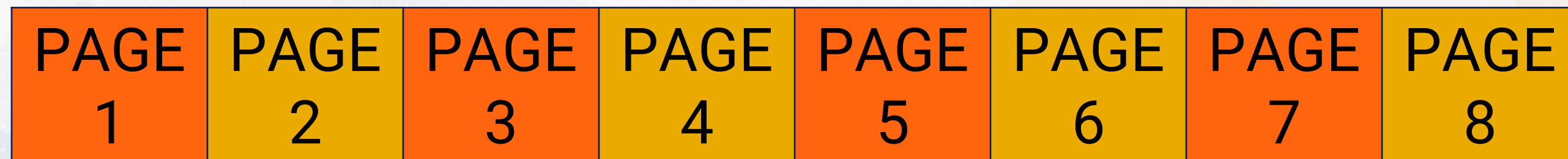
MAX_PAGES_IN_MEMORY

BUFFER POOL

PAGE 1	PAGE 2	PAGE 3	PAGE 4
-----------	-----------	-----------	-----------

MAX_PAGES_IN_MEMORY

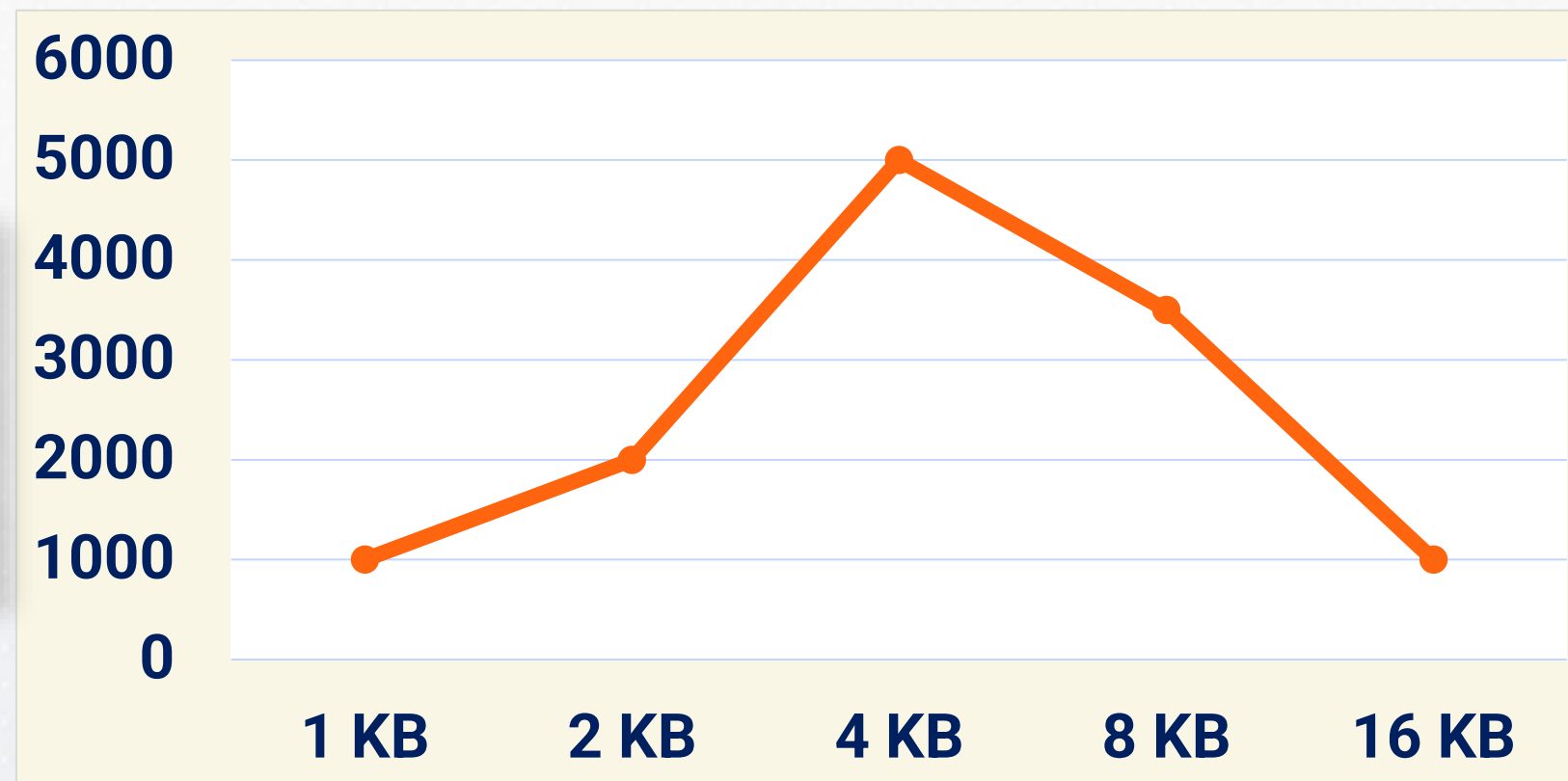
BUFFER POOL



Challenges in Configuration Tuning

Most parameters like **PAGE_SIZE** have a “sweet spot”

THROUGHPUT
(TRANSACTIONS
PER SECOND)



PAGE SIZE PARAMETER

Challenges in Configuration Tuning

PostgreSQL's default configuration has **low** buffer pool size

System Parameter	Value
BUFFER_POOL_SIZE	128 MB



System Parameter	Value
BUFFER_POOL_SIZE	16 GB

Immutable vs. Mutable Parameters

System Parameter	Mutable or Immutable
PAGE_SIZE	Immutable
BUFFER_POOL_SIZE	Mutable

Cloud Database Tuning

Cost vs Performance

Memory Size	Throughput	Cost	Price-Performance
8 GB	100 transactions per second	\$0.05 per hour	\$0.139 per million transactions
16 GB	200 transactions per second	\$0.15 per hour	\$0.208 per million transactions

Tools for Configuration Tuning

Automatically recommend configuration
based on hardware and application

Parameter	Value
TOTAL_MEMORY	16 GB
NUMBER_OF_CPUS	16
STORAGE_TYPE	SSD / HDD / SAN
APPLICATION_TYPE	Type 1 / Type 2 / Type 3

Database Application Type

Application Type	Performance Bound by	Database Size Relative to DRAM	Workload Characteristics
Online Transaction Processing (OLTP)	CPU- or I/O-bound	DB slightly larger than DRAM ~ 1 TB	20-40% short transactions, some long transactions and queries
Online Analytical Processing (OLAP)	I/O- or DRAM-bound	DB much larger than DRAM ~ 4 TB	Large complex reporting queries; Also known as data warehouse
Web Application (WEB)	CPU-bound	DB much smaller than DRAM ~ 4 GB	90% or more simple queries

Reinforcement Learning for Configuration Tuning

- Agent interacts with database by trying different configuration settings
- Observes the impact on performance such as throughput

State	Current settings of database parameters and the current workload characteristics
Action	Increase or decrease the value of some parameter
Reward	Increase in throughput or decrease in price-performance metric



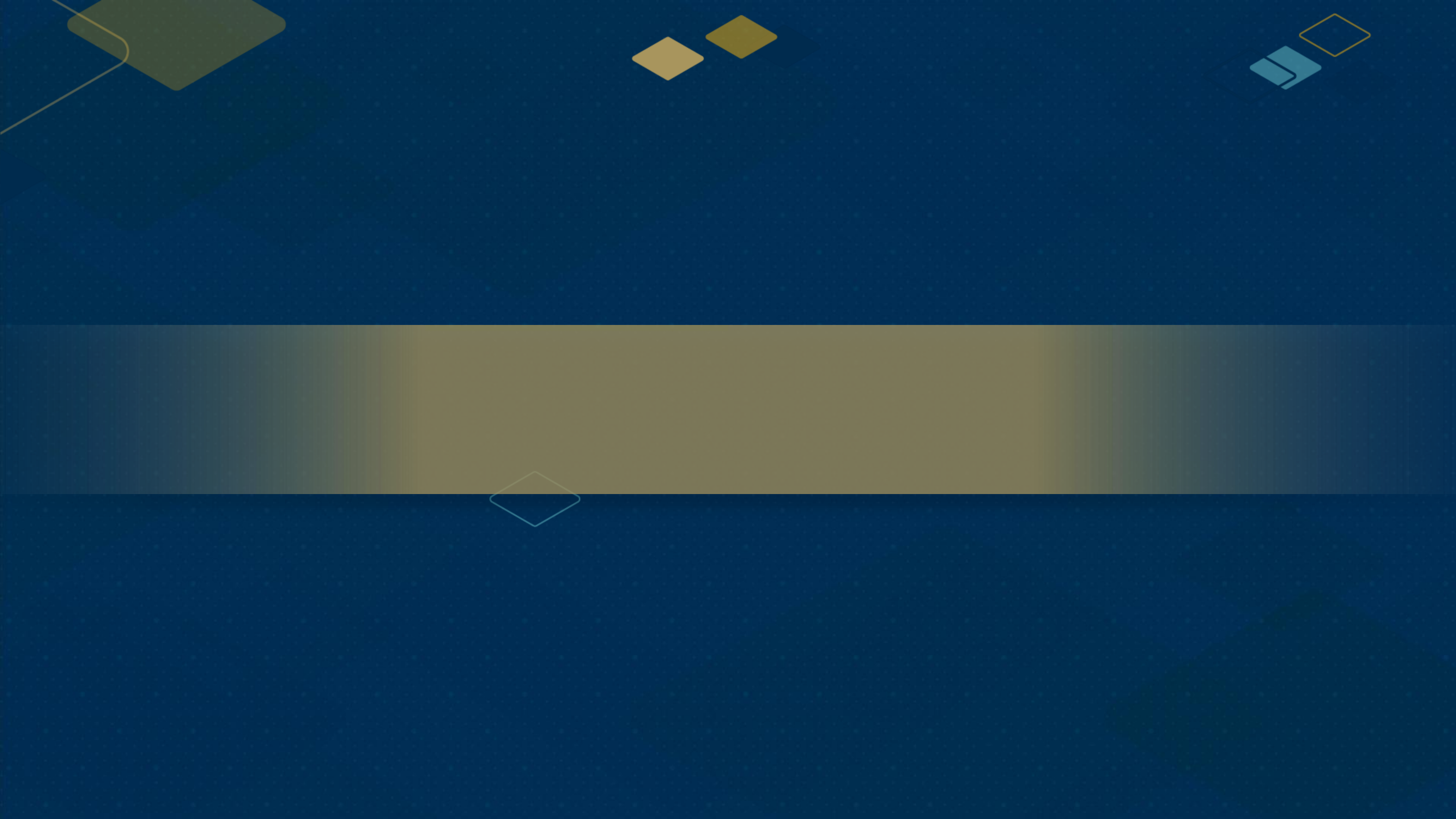


Database Configuration



**Database
Configuration**

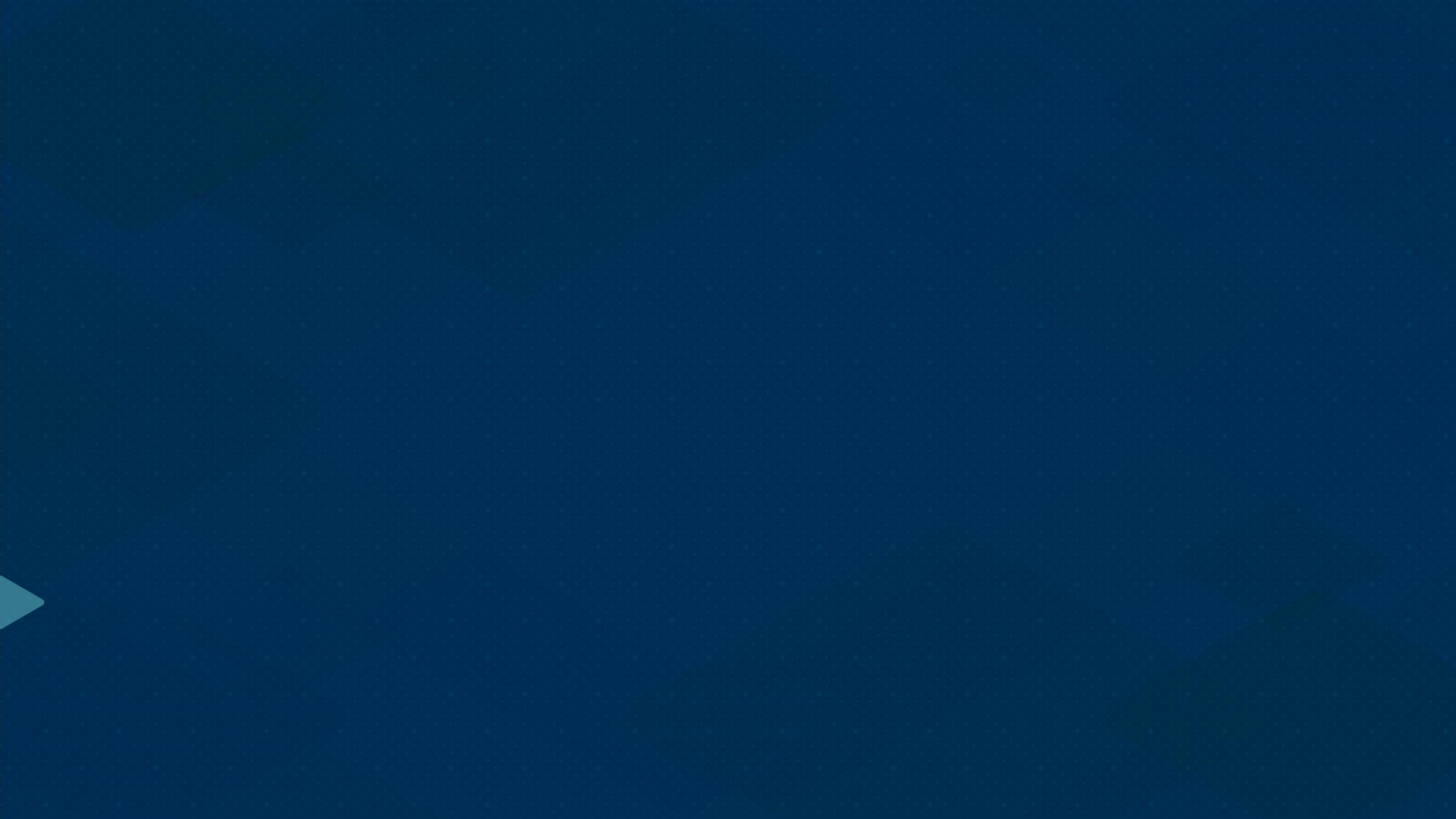
**OLTP
and OLAP**





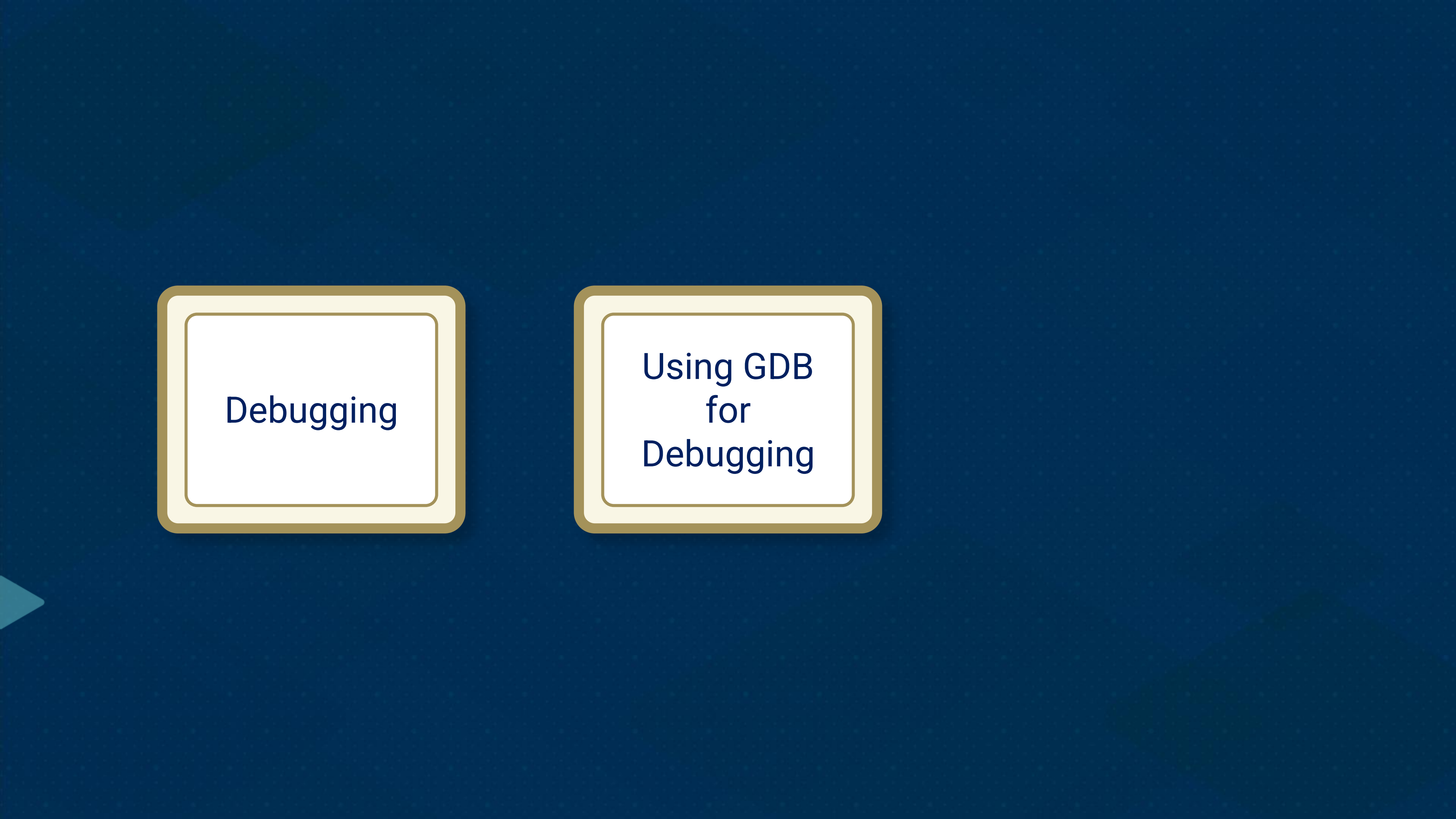
Database System Debugging







Debugging



Debugging

Using GDB
for
Debugging

Debugging

Using GDB
for
Debugging

Debugging
Principles

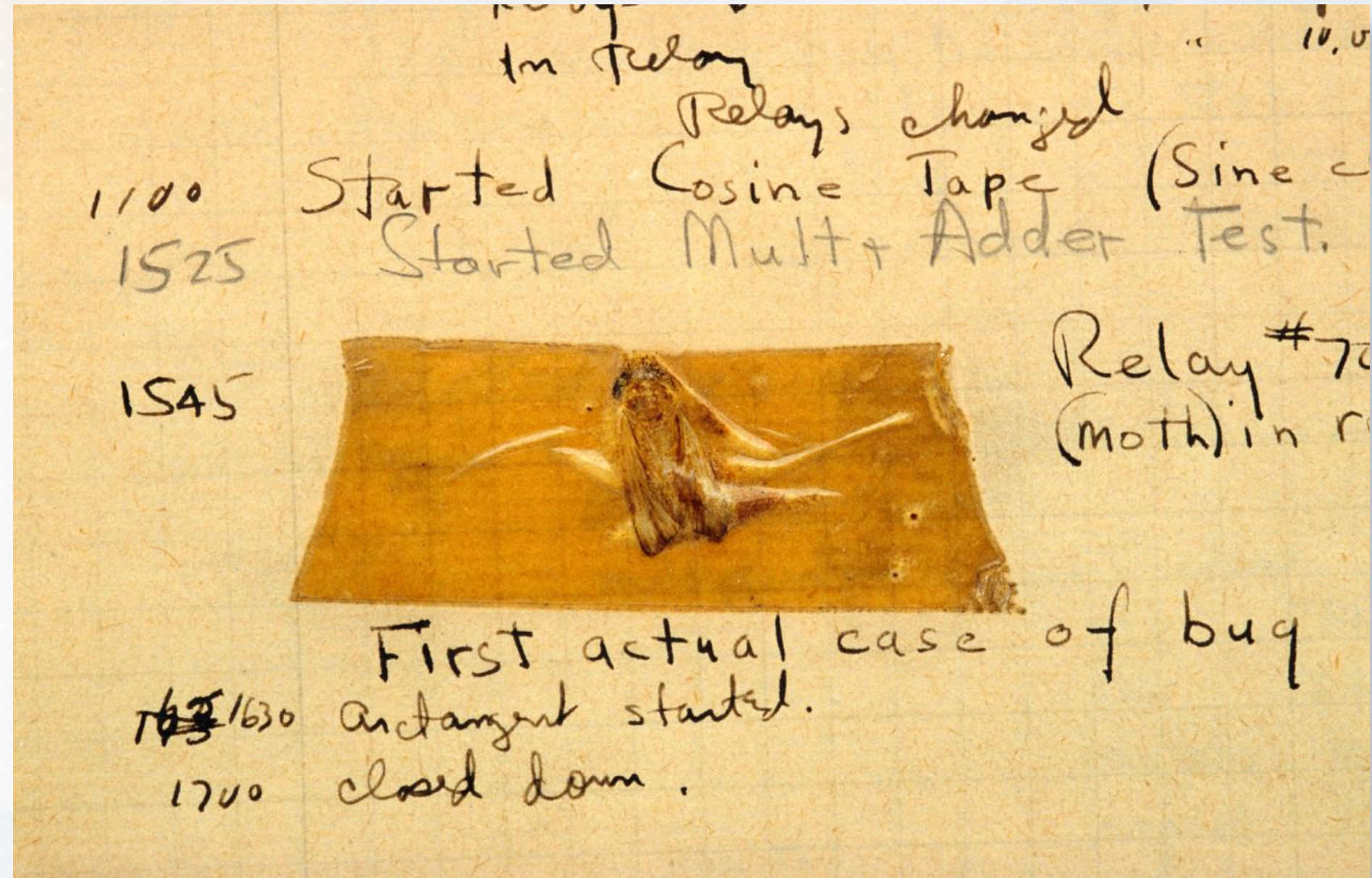
Origin of Debugging



GRACE HOPPER

James S. Davis

Public domain, via Wikimedia Commons



Log Book with Computer Bug

National Museum of American History

Public Domain under Creative Commons Zero (CC0) license

Debugging in Database Systems



Bugs

Data Corruption

Performance
Degradation

System Crashes



Tools

GDB

Print Statements

Using GDB for Debugging

GDB

a tool to see what is going on 'inside' the program while it executes or at the moment it crashed

```
g++ -g buzzdb.cpp -o buzzdb  
gdb ./buzzdb
```

Using GDB for Debugging

```
#include <iostream>

int add(int x, int y) {
    return x + y; // Set a breakpoint here
}

int main() {
    int sum = 0;
    for (int i = 1; i <= 10; ++i) {
        sum = add(sum, i);
        std::cout << "Sum: " << sum << endl;
    }
    return 0;
}
```

GDB Commands

GDB Commands

Run:
Starts the
program
under GDB

GDB Commands

Run:

Starts the
program
under GDB

Next:

Moves
Execution
to the next
line

GDB Commands

Run:
Starts the
program
under GDB

Next:
Moves
Execution
to the next
line

Print:
Displays
values of
variables

GDB Commands

Run:
Starts the
program
under GDB

Next:
Moves
Execution
to the next
line

Print:
Displays
values of
variables

Break: Set a
breakpoint
at a specific
line or
function

GDB Commands

GDB Commands

Continue:

Continue running
the program until
the next
breakpoint

GDB Commands

Continue:

Continue running the program until the next breakpoint

Backtrace (bt):

Shows the call stack to see how the program reached current point

GDB Commands

Continue:

Continue running the program until the next breakpoint

Backtrace (bt):

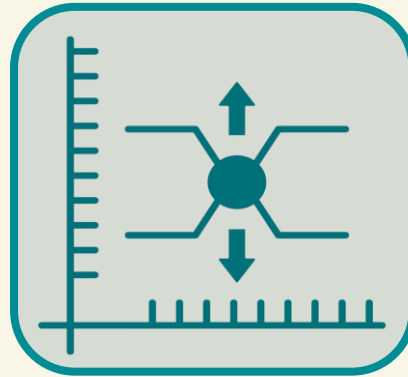
Shows the call stack to see how the program reached current point

Info:

Displays local variables in the current stack frame

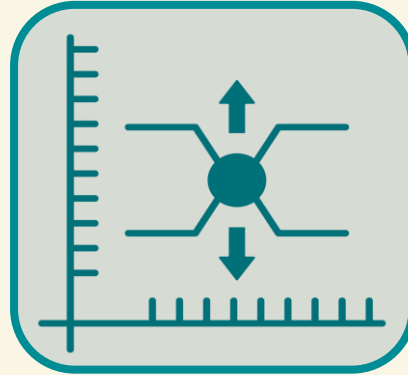
Breakpoints and Backtrace

Breakpoints and Backtrace

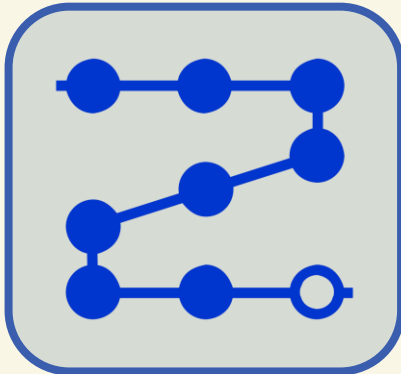


Breakpoints: Temporarily halt the program execution at a specific point

Breakpoints and Backtrace



Breakpoints: Temporarily halt the program execution at a specific point



Backtrace: Reveals the path taken by the program to reach current execution point

Illustrative GDB Session

```
(gdb) break add
```

```
Breakpoint 1 at 0x...: file main.cpp, line 4.
```

```
(gdb) run
```

```
Starting program: /a.out
```

```
Breakpoint 1, add (x=0, y=1) at main.cpp:4
```

```
4    return x + y;
```

```
(gdb) info locals
```

```
x = 0
```

```
y = 1
```

```
(gdb) next
```

```
5 }
```

Illustrative GDB Session

```
(gdb) print x
```

```
$1 = 0
```

```
(gdb) print y
```

```
$2 = 1
```

```
(gdb) continue
```

```
Continuing.
```

```
Sum: 1
```

```
(gdb) backtrace
```

```
#0 add (x=1, y=2) at main.cpp:4
```

```
#1 0x... in main () at main.cpp:8
```

Using Print Statements for Debugging

Print statements allow you to track how your program's execution flow and how variables change over time

```
std::cout << "Loading page : " << page_id << std::endl;  
std::cout << "Evicting page: " << evictedPageId << std::endl;
```

Overload << operator

Print statements allow you to track how your program's execution flow and how variables change over time

```
// Define the operator<< function
std::ostream &operator<<(std::ostream &os, const Person &person) {
    os << "Person[name=" << person.name << ", age=" << person.age << "]\n";
    return os;
}

int main() {
    Person alice("Alice", 50);
    std::cout << alice << std::endl;
}
```

OUTPUT: Person[name = Alice, age = 50]

Debugging Principle: Stress Testing

```
constexpr size_t MAX_PAGES_IN_MEMORY = 10; // For stress testing  
BufferManager bufferManager(MAX_PAGES_IN_MEMORY);
```

Debugging Principle: Stress Testing

Small Buffer Pool

makes it easier
to observe and
understand
eviction
decisions

```
constexpr size_t MAX_PAGES_IN_MEMORY = 10; // For stress testing  
BufferManager bufferManager(MAX_PAGES_IN_MEMORY);
```

Debugging Principle: Stress Testing

Small Buffer Pool

makes it easier
to observe and
understand
eviction
decisions

Limited Slots

edge-cases such
as buffer pool
arise much
sooner

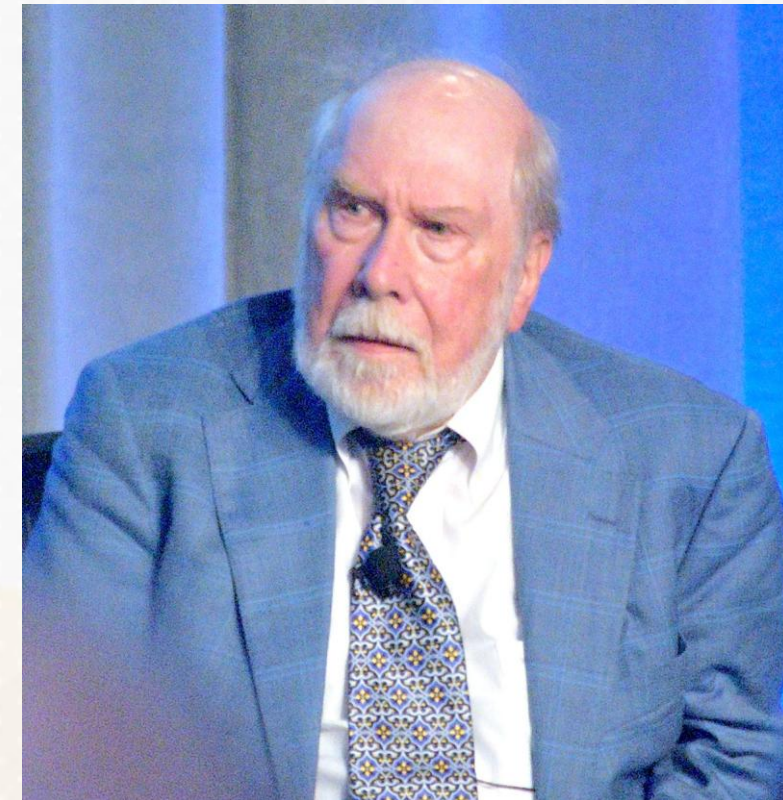
```
constexpr size_t MAX_PAGES_IN_MEMORY = 10; // For stress testing  
BufferManager bufferManager(MAX_PAGES_IN_MEMORY);
```

Debugging Principle: Modular Design

Modularity

clear interfaces and separation of concerns

```
class BufferManager {  
    StorageManager storage_manager;  
    std::unique_ptr<Policy> policy;  
    ...  
};
```



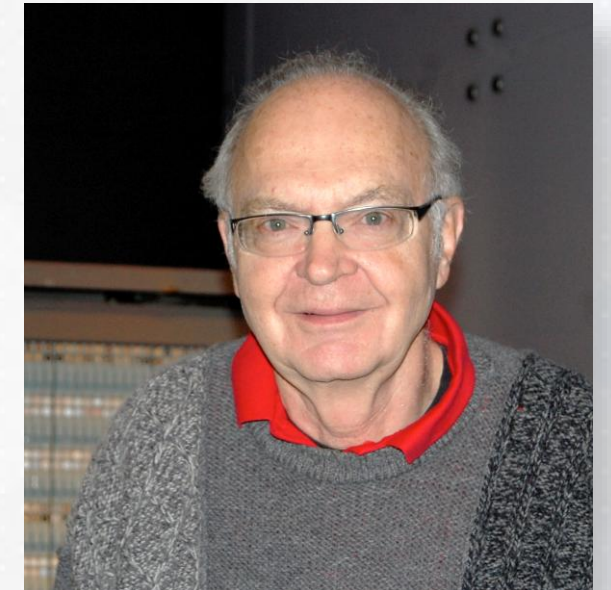
NIKLAUS WIRTH

orcmid, CC BY 2.0
via Wikimedia Commons

Debugging Principle: Literate Programming

programs must be understandable by humans first.
documentation is an integral part of programming.

```
/**
 * Function: readPage
 * Purpose: Reads a page from the buffer pool.
 *          If the page is not currently in memory,
 *          it loads the page from disk into the buffer pool.
 * Inputs: pageId - The ID of the page to be read
 * Outputs: A reference to a Buffer containing the page data
 * Throws: DiskIOException if the page cannot be read
 */
Buffer &BufferManager::readPage(PageId pageId) {
    ...
}
```



DONALD KNUTH

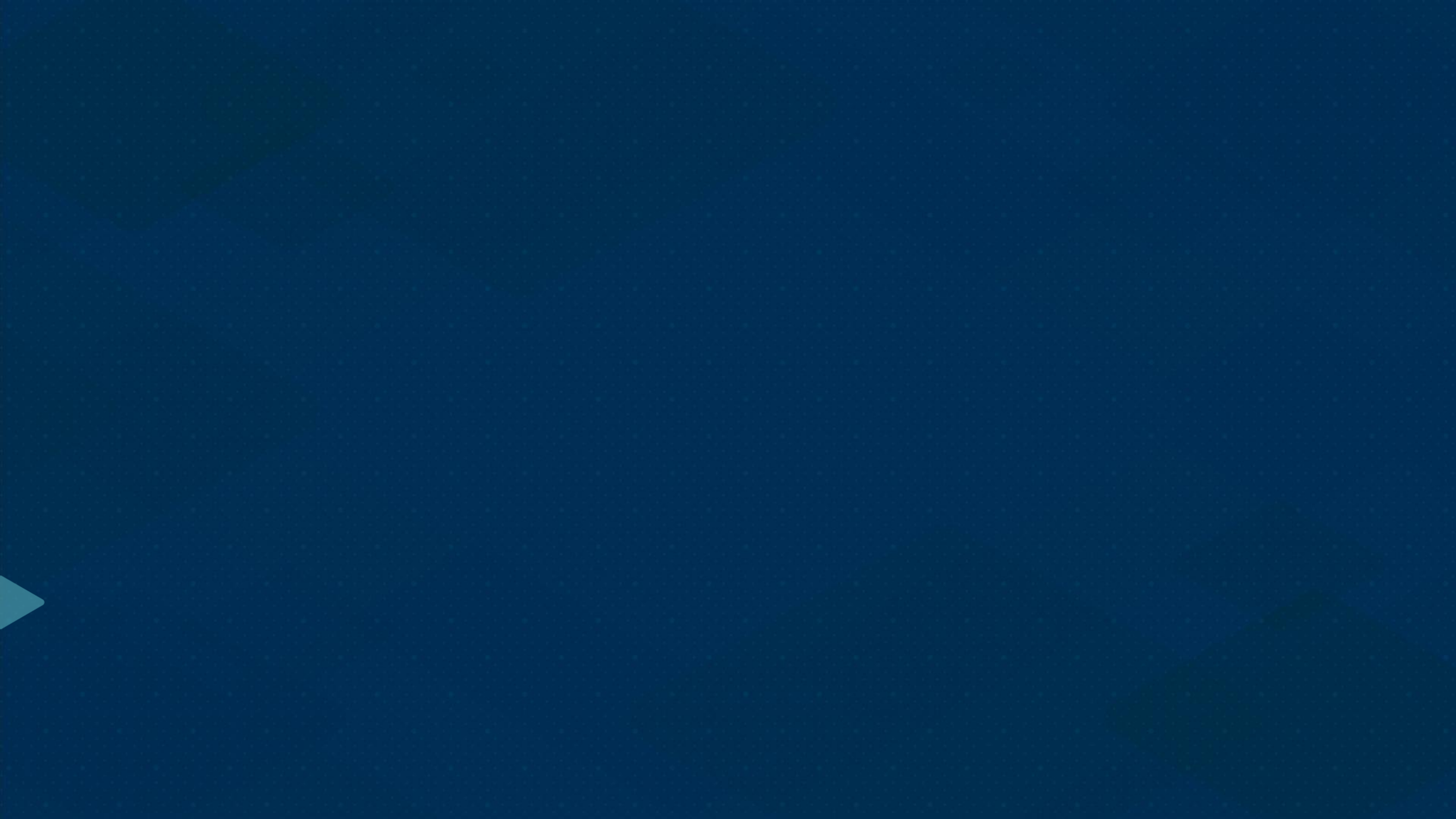
Alex Handy, CC BY-SA 2.0
via Wikimedia Commons

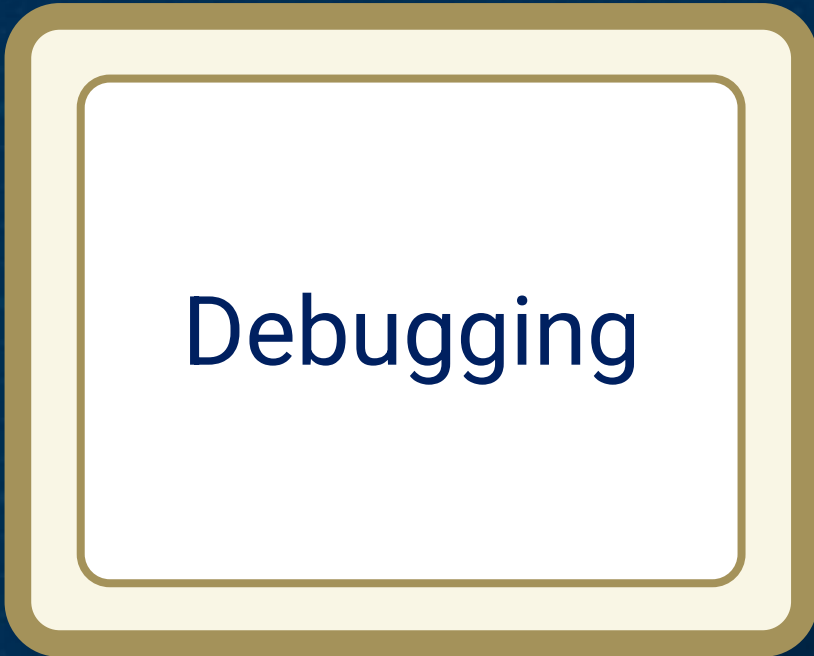
Debugging Principle: Robustness

Robustness

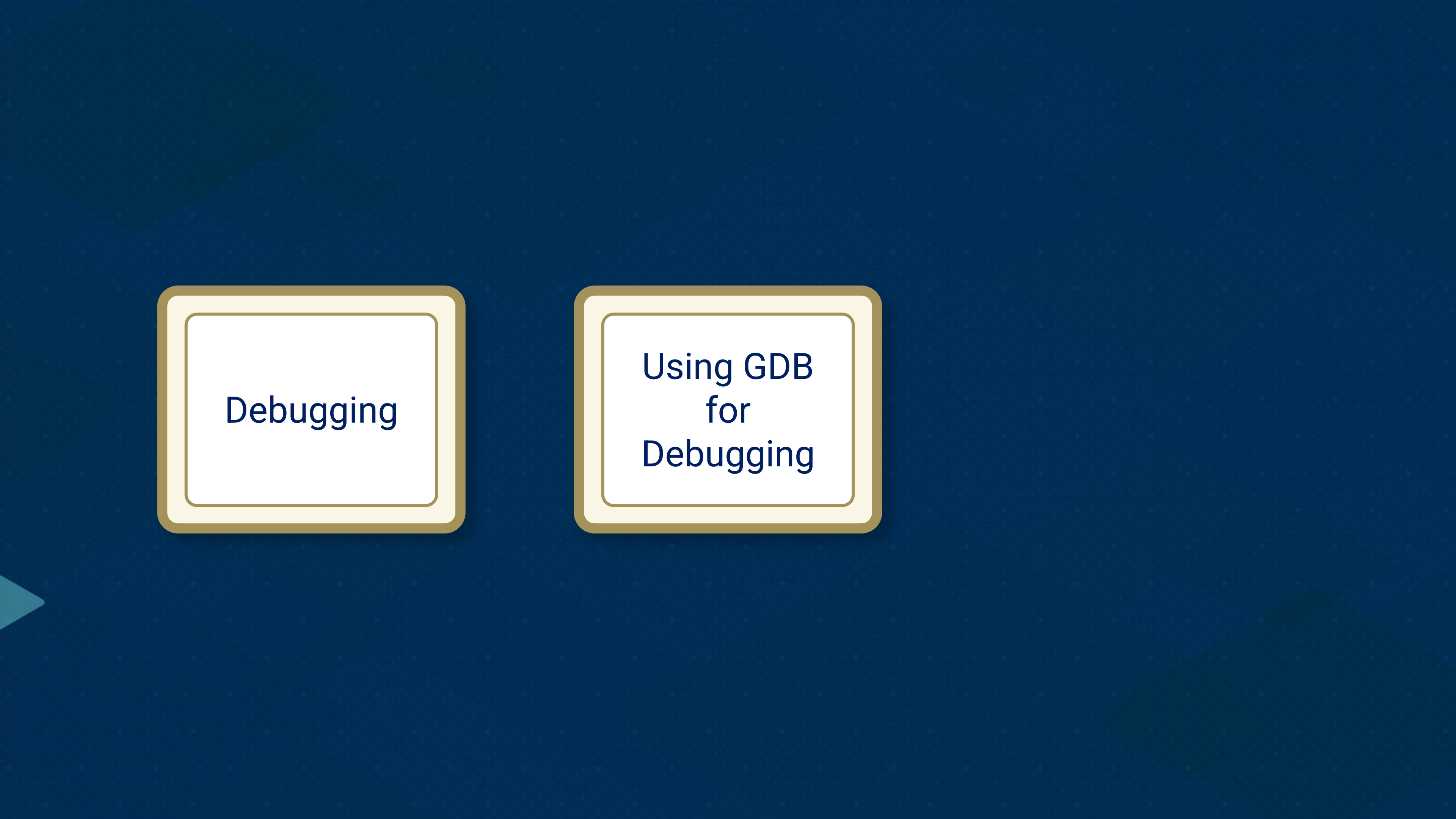
ensure that the system can withstand failures, like failed disk I/O operations

```
// Load frame from disk into memory bool
BufferManager::loadFrame(int frameId, int pageId) {
    try {
        // Call the storage manager to read data from disk into the buffer frame
        storageManager.readPage(pageId, pool[frameId].data);
    } catch (const DiskIOException &e) {
        std::cerr << "Failed to load page from disk: " << e.what() << std::endl;
        return false;
    }
    return true;
}
```





Debugging



Debugging

Using GDB
for
Debugging

Debugging

Using GDB
for
Debugging

Debugging
Principles



Debugging Principles



Debugging Principle: Stress Testing

```
constexpr size_t MAX_PAGES_IN_MEMORY = 3; // For testing  
BufferManager bufferManager(MAX_PAGES_IN_MEMORY);
```

Debugging Principle: Stress Testing

Small Buffer Pool

makes it easier
to observe and
understand
eviction
decisions

```
constexpr size_t MAX_PAGES_IN_MEMORY = 3; // For testing  
BufferManager bufferManager(MAX_PAGES_IN_MEMORY);
```

Debugging Principle: Stress Testing

Small Buffer Pool

makes it easier
to observe and
understand
eviction
decisions

Limited Slots

edge-cases such
as buffer pool
arise much
sooner

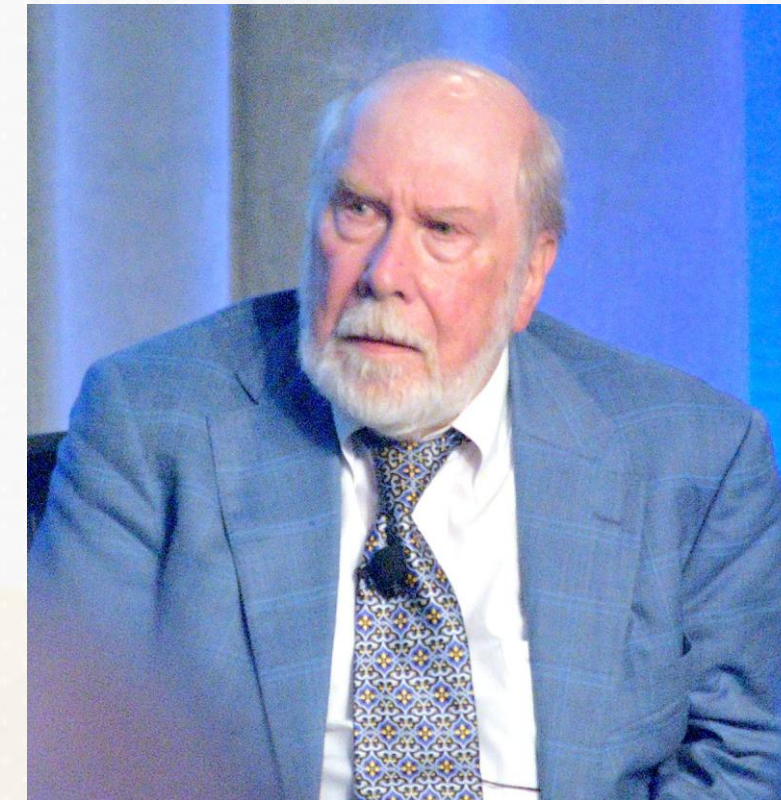
```
constexpr size_t MAX_PAGES_IN_MEMORY = 3; // For testing  
BufferManager bufferManager(MAX_PAGES_IN_MEMORY);
```

Debugging Principle: Modular Design

Modularity

clear interfaces and separation of concerns, such as separating management of buffer metadata from the actual data storage

```
class BufferPool {  
    MetadataManager metadata;  
    DataManager data;  
  
public:  
    Buffer& getBuffer(int id) {  
        metadata.updateAccessTime(id);  
        return data.getBufferData(id);  
    }  
};
```



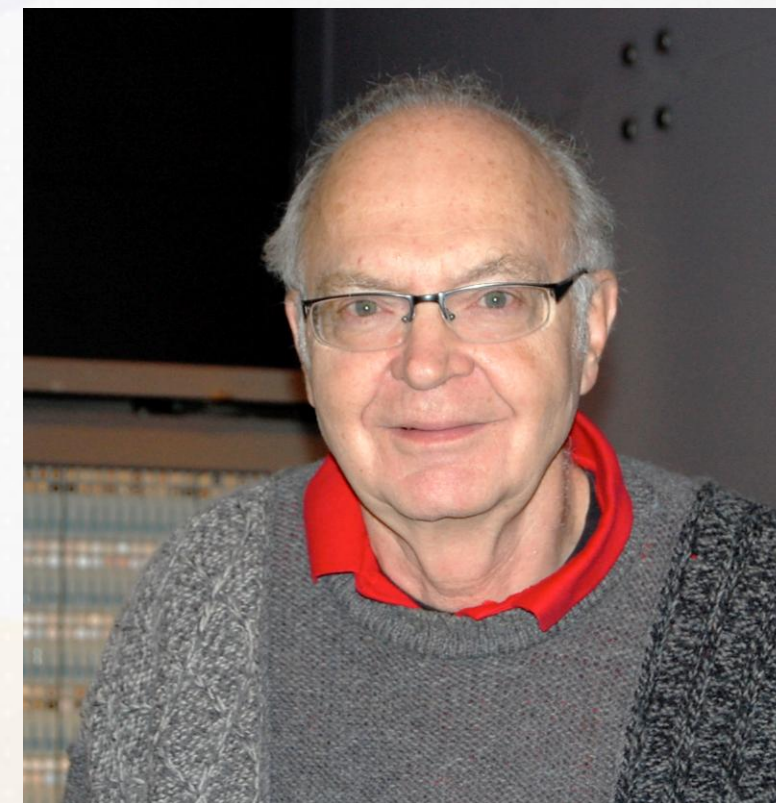
NIKLAUS WIRTH

orcmid, CC BY 2.0
via Wikimedia Commons

Debugging Principle: Literate Programming

Write programs that are understandable by humans first, treating documentation as an integral part of programming.

```
/**
 * Function: allocateBuffer
 * Purpose: Allocates a buffer from the pool.
 *          If no buffers are available,
 *          it evicts the least recently used (LRU) buffer.
 * Inputs: None
 * Outputs: A reference to a Buffer
 * Throws: BufferAllocationException if
 *          no buffers can be freed.
 */
Buffer& BufferPool::allocateBuffer() {
    ...
}
```



DONALD KNUTH

Alex Handy, CC BY-SA 2.0
via Wikimedia Commons

Debugging Principle: Robustness

Robustness

ensuring that the system can withstand failures, like failed disk I/O operations

```
bool BufferPool::loadBuffer(int bufferId, int diskSector) {  
    try {  
        diskManager.readData(diskSector, buffers[bufferId].data);  
    } catch (const DiskIOException& e) {  
        std::cerr << "Failed to load buffer from disk: "  
        << e.what() << std::endl;  
        return false;  
    }  
    return true;  
}
```